

CURSO INTENSIVO · PYTHON 3.12+

Autor: Claude (Anthropic) · **Data:** 29 de julho de 2026 · **Público:** profissionais técnicos
(redes, infra, segurança, automação)

Sumário

Módulo 1 — Mentalidade Python para quem já é técnico	3
Módulo 2 — Setup profissional (15 minutos)	6
Módulo 3 — Fundamentos que realmente importam	8
Módulo 4 — Estruturas de dados essenciais	12
Módulo 5 — Strings, datas e JSON	15
Módulo 6 — Arquivos e automação	18
Módulo 7 — Tratamento de erros profissional	22
Módulo 8 — Orientação a objetos sem complicação	25
Módulo 9 — Tipagem moderna (MUITO IMPORTANTE)	28
Módulo 10 — HTTP do zero ao profissional	31
Módulo 11 — APIs REST na prática	34
Módulo 12 — FastAPI (núcleo do curso)	37
Módulo 13 — Pydantic v2	42
Módulo 14 — Banco de dados com SQLAlchemy	45
Módulo 15 — AsyncIO para quem tem lógica forte	49
Módulo 16 — Autenticação e segurança	52
Módulo 17 — Testes que importam	55
Módulo 18 — Logs, observabilidade e debugging	58
Módulo 19 — Docker para APIs Python	61
Módulo 20 — Deploy rápido	63
Módulo 21 — Bibliotecas que você realmente usará	65
Módulo 22 — Integrações reais (o mais importante)	66
Módulo 23 — Projeto final: Sistema de Cobranças e Lembretes	72
Módulo 24 — Plano de estudo de 7 dias (2h30/dia)	76
Módulo 25 — Cheatsheets finais	77
Apêndice A — Erros mais comuns de iniciantes técnicos	79
Apêndice B — Estrutura de projeto recomendada para produção	80
Apêndice C — Como ler documentação Python rapidamente	81

Módulo 1 — Mentalidade Python para quem já é técnico

Você já domina lógica, redes, Linux, Docker e banco de dados. Este módulo existe para recalibrar sua cabeça — não para ensinar o que é uma variável.

O que muda em relação a C#, JavaScript e shell script

Vindo de...	O que muda no Python
C#	Sem compilação, sem <code>{ }</code> (indentação define blocos), sem tipos obrigatórios. Tipagem é dinâmica, mas <i>type hints</i> recuperam boa parte da segurança do C#.
JavaScript	Sem <code>===</code> vs <code>==</code> (Python só tem <code>==</code> sensato), sem <code>callback hell</code> — <code>async</code> é explícito com <code>async/await</code> . Sem hoisting, sem <code>undefined</code> . Existe <code>None</code> , e só ele.
Shell script	Você para de fazer parsing de texto com <code>awk/sed</code> e passa a trabalhar com estruturas de dados reais (dict, list). Subprocessos existem (<code>subprocess</code>), mas raramente são necessários.

Diferenças práticas que pegam todo mundo:

```
# Blocos por indentação (4 espaços, sempre)
if status == "ativo":
    enviar_cobranca(cliente)      # dentro do if
enviar_relatorio()              # fora do if – a indentação É a sintaxe

# Booleanos são capitalizados
ativo = True                    # não "true"
vazio = None                    # não "null" nem "nil"

# Não existe ++ nem --
contador += 1
```

Tipagem dinâmica vs tipagem estática

Python decide o tipo em tempo de execução:

```
payload = {"agente": "srv-web-01"} # dict
payload = "srv-web-01"             # agora é str – nenhum erro
```

Isso acelera prototipagem, mas em produção você **sempre** usará *type hints* (Módulo 9) + Pydantic (Módulo 13) para validar dados na fronteira do sistema (APIs, arquivos, webhooks). A regra profissional em 2026 é:

Código dinâmico por dentro, validação rígida nas bordas.

Duck typing

"Se anda como pato e grasna como pato, é pato." Python não pergunta o tipo do objeto — pergunta se ele tem o método necessário:

```
def salvar_log(destino):
    destino.write("evento registrado\n")    # funciona com arquivo, socket,
                                           # StringIO, resposta HTTP...
```

Qualquer objeto com método `.write()` serve. Isso é a base de por que bibliotecas Python se integram tão bem entre si.

Zen of Python

Rode `python3 -c "import this"`. Os princípios que importam na prática:

- **Explícito é melhor que implícito** — nada de mágica escondida.
- **Simple é melhor que complexo** — se precisa de um diagrama para explicar, refatore.
- **Erros nunca devem passar silenciosamente** — trate exceções específicas (Módulo 7).
- **Deve haver um — e preferencialmente só um — modo óbvio de fazer algo.**

Como escrever código Python "profissional"

1. **snake_case** para funções e variáveis, **PascalCase** para classes, **MAIÚSCULAS** para constantes.
2. Use `ruff` (linter + formatter, substituiu flake8/black em velocidade) desde o primeiro dia.
3. Funções curtas, com type hints e um único propósito.
4. `if __name__ == "__main__":` para separar código executável de importável.
5. Nunca capture `except Exception` sem re-lançar ou logar.

```
# Padrão mínimo de um script profissional
import logging

logger = logging.getLogger(__name__)

def processar_agentes(caminho: str) -> int:
    """Processa lista de agentes e retorna quantos foram atualizados."""
    ...

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    total = processar_agentes("agentes.csv")
    logger.info("Processados: %d", total)
```

O que evitar logo no início

- `from modulo import *` — polui o namespace e esconde a origem das funções.
- **Variáveis globais mutáveis** — passe parâmetros; use classes ou injeção de dependência.
- `except:` **genérico** — engole `KeyboardInterrupt` e bugs reais.
- **Concatenar strings com `+` em loop** — use `"".join()` ou f-strings.
- **Argumentos default mutáveis:** `def f(itens=[])` — o mesmo objeto lista é compartilhado entre todas as chamadas. Use `itens: list | None = None`.
- **Reinventar a roda** — antes de escrever, verifique se já existe na stdlib (`pathlib`, `json`, `csv`, `datetime`, `itertools`).
- **Instalar pacotes globalmente** — sempre venv (Módulo 2).

Módulo 2 — Setup profissional (15 minutos)

Instalação do Python

Linux (Ubuntu/Debian):

```
sudo apt update && sudo apt install -y python3.12 python3.12-venv python3-pip
python3 --version # Python 3.12.x
```

Windows: baixe em python.org e marque **"Add Python to PATH"**. Verifique com `py --version`.

VS Code + extensões essenciais

Instale apenas estas (2026):

Extensão	Função
Python (Microsoft)	Suporte base, debug, venv
Pylance	Autocomplete + checagem de tipos em tempo real
Ruff	Lint + format automático ao salvar
Even Better TOML	Editar <code>pyproject.toml</code>

Configuração recomendada (`settings.json`):

```
{
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff"
  },
  "python.analysis.typeCheckingMode": "basic"
}
```

Virtual environments (venv)

Regra de ouro: um projeto = um venv. Nunca instale dependências no Python do sistema.

```
mkdir api-cobrancas && cd api-cobrancas
python3 -m venv .venv

# Ativar
source .venv/bin/activate # Linux/macOS
.venv\Scripts\activate    # Windows

# Confirmar que está isolado
which python # ../api-cobrancas/.venv/bin/python
```

O VS Code detecta o `.venv` automaticamente (canto inferior direito → selecione o interpretador).

pip e requirements.txt

```
pip install fastapi uvicorn httpx

# Congelar versões exatas do ambiente
pip freeze > requirements.txt

# Restaurar em outra máquina/container
pip install -r requirements.txt
```

requirements.txt típico:

```
fastapi==0.115.8
uvicorn[standard]==0.34.0
httpx==0.28.1
```

pyproject.toml (introdução)

O padrão moderno para projetos que serão versionados/empacotados. Mesmo em projetos simples, já serve para configurar ferramentas:

```
[project]
name = "api-cobrancas"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115",
    "httpx>=0.28",
]

[tool.ruff]
line-length = 100

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Dica 2026: conheça o `uv` (da Astral, mesma equipe do Ruff). Ele substitui pip + venv com resolução de dependências ~50x mais rápida: `uv venv && uv pip install -r requirements.txt`. Nos exemplos do curso usamos pip por ser universal.

Executando scripts e módulos

```
python script.py           # executa um arquivo
python -m app.main         # executa como módulo (respeita imports do pacote)
python -m http.server 8080 # módulos da stdlib são executáveis
python -i script.py        # executa e abre shell interativo (ótimo p/ debug)
```

Use `python -m` sempre que o código estiver dentro de um pacote — evita 90% dos erros de `ModuleNotFoundError` por import relativo.

Módulo 3 — Fundamentos que realmente importam

Somente o necessário para backend. Sem teoria de sobra.

Variáveis e tipos básicos

```
timeout = 30          # int
taxa = 0.15           # float
host = "10.0.30.11"  # str
ativo = True          # bool
resposta = None       # ausência de valor
```

Não há declaração de tipo, mas **anote os tipos** (hábito desde já):

```
timeout: int = 30
host: str = "10.0.30.11"
```

Conversões

```
porta = int("1514")          # str -> int (lança ValueError se inválido)
valor = float("199.90")
texto = str(500)
bool("")                     # False – string vazia é "falsy"
bool("qualquer coisa")      # True

# Valores "falsy": None, 0, "", [], {}, set()
if not agentes:              # forma idiomática de testar lista vazia
    print("nenhum agente retornado")
```

f-strings

O único jeito de formatar strings que você precisa saber:

```
agente = "srv-db-02"
cpu = 87.3456

print(f"Agente {agente} com CPU em {cpu:.1f}%")  # casas decimais
print(f"{'ID':<8}{'STATUS':<10}")                # alinhamento (tabelas)
print(f"{1234567:,}".replace(",", "."))         # 1.234.567
print(f"cpu={cpu}")                             # cpu=87.3456 (debug!)
```

Condicionais

```

if status_code == 200:
    processar(resposta)
elif status_code in (401, 403):
    renovar_token()
elif 500 <= status_code < 600:
    agendar_retry()
else:
    logger.warning("Status inesperado: %s", status_code)

# Ternário
ambiente = "prod" if host.startswith("srv-") else "lab"

# match/case (Python 3.10+) – ótimo para eventos de webhook
match evento["tipo"]:
    case "pagamento.aprovado":
        baixar_titulo(evento)
    case "pagamento.recusado" | "pagamento.expirado":
        notificar_cliente(evento)
    case _:
        logger.info("Evento ignorado: %s", evento["tipo"])

```

Loops

```

# for percorre coleções diretamente – sem índice manual
for agente in agentes:
    verificar(agente)

# Se precisar do índice
for i, agente in enumerate(agentes, start=1):
    print(f"{i}/{len(agentes)}: {agente}")

# Dois iteráveis em paralelo
for host, ip in zip(hosts, ips):
    registrar_dns(host, ip)

# while com retry – padrão comum em integrações
tentativas = 0
while tentativas < 3:
    try:
        resposta = chamar_api()
        break
    except TimeoutError:
        tentativas += 1
else:
    # executa se o while terminar SEM break
    raise RuntimeError("API indisponível após 3 tentativas")

```

Funções, escopo e argumentos

```
def buscar_agentes(status: str = "active", limite: int = 100) -> list[dict]:
    """Busca agentes na API do gerenciador."""
    ...

# Argumentos nomeados: legibilidade em chamadas com muitos parâmetros
buscar_agentes(status="disconnected", limite=500)
```

Escopo: variáveis criadas dentro da função morrem com ela. Funções enxergam (mas não modificam) variáveis do módulo. Evite `global` — passe parâmetros.

*args e **kwargs

```
def montar_url(base: str, *segmentos: str, **params: str) -> str:
    """montar_url("https://api.x.com", "v1", "agents", status="active")
    -> https://api.x.com/v1/agents?status=active"""
    caminho = "/".join(segmentos)
    query = "&".join(f"{k}={v}" for k, v in params.items())
    return f"{base}/{caminho}" + (f"?{query}" if query else "")
```

Uso real: repassar parâmetros para outra função sem redeclarar todos:

```
def get_com_log(url: str, **kwargs):
    logger.info("GET %s", url)
    return httpx.get(url, **kwargs)    # timeout, headers, etc. passam direto
```

Retorno múltiplo

```
def validar_payload(dados: dict) -> tuple[bool, str]:
    if "cliente_id" not in dados:
        return False, "cliente_id ausente"
    return True, ""

ok, erro = validar_payload(payload)    # desempacotamento
if not ok:
    raise ValueError(erro)
```

Exercício: parser de parâmetros HTTP

Implemente `parse_query(qs: str) -> dict[str, str]` que transforme `"status=active&os=windows&limit=50"` em `{"status": "active", "os": "windows", "limit": "50"}`.

Solução:

```
def parse_query(qs: str) -> dict[str, str]:
    if not qs:
        return {}
    pares = qs.split("&")
    resultado: dict[str, str] = {}
    for par in pares:
        chave, _, valor = par.partition("=")
        resultado[chave] = valor
    return resultado

assert parse_query("status=active&limit=50") == {"status": "active", "limit": "50"}
```

Versão em uma linha (após o Módulo 4 você entenderá):

```
def parse_query(qs: str) -> dict[str, str]:
    return dict(p.partition("=")[::2] for p in qs.split("&")) if qs else {}
```

Módulo 4 — Estruturas de dados essenciais

80% do trabalho backend é transformar dict e list. Domine isso e o resto flui.

list — sequência mutável

```
hosts = ["srv-web-01", "srv-db-02"]
hosts.append("srv-app-03")
hosts.extend(["srv-log-04", "srv-cache-05"])
hosts[0]           # primeiro
hosts[-1]          # último
hosts[1:3]         # fatia (índices 1 e 2)
"srv-db-02" in hosts # True – teste de pertencimento
len(hosts)
```

tuple — sequência imutável

Use para dados que não devem mudar (coordenadas, pares chave-valor, retorno múltiplo):

```
credencial = ("wazuh-api", "s3cr3t")
usuario, senha = credencial # desempacotamento
```

dict — o coração do backend

Todo JSON vira dict. Toda linha de banco vira dict. Todo payload é dict.

```
agente = {"id": "017", "name": "srv-web-01", "status": "active"}

agente["ip"] = "10.0.30.11" # adicionar/atualizar
agente.get("os")           # None se não existir (não lança erro)
agente.get("os", "desconhecido") # com default
agente.pop("status", None)  # remove com segurança

for chave, valor in agente.items():
    print(f"{chave}: {valor}")

# Mesclar dicts (3.9+) – ex.: defaults + overrides de config
config = {"timeout": 30, "verify": True} | {"timeout": 60}
# {"timeout": 60, "verify": True}
```

set — conjunto sem duplicatas

Perfeito para comparar inventários:

```
agentes_cadastrados = {"srv-web-01", "srv-db-02", "srv-app-03"}
agentes_reportando = {"srv-web-01", "srv-app-03", "srv-novo-09"}

agentes_cadastrados - agentes_reportando # {'srv-db-02'} – sumiu do monitoramento
agentes_reportando - agentes_cadastrados # {'srv-novo-09'} – não cadastrado
agentes_cadastrados & agentes_reportando # interseção
```

Compreensões (comprehensions)

A ferramenta mais usada do Python profissional:

```
# List comprehension: transformar
nomes = [a["name"] for a in agentes]

# Com filtro
desconectados = [a for a in agentes if a["status"] == "disconnected"]

# Dict comprehension: reindexar por chave
por_id = {a["id"]: a for a in agentes}
por_id["017"]["name"] # acesso O(1), sem varrer a lista

# Set comprehension: valores únicos
sistemas = {a["os"]["platform"] for a in agentes if "os" in a}
```

Ordenação

```
# sorted() retorna nova lista; key define o critério
agentes_ord = sorted(agentes, key=lambda a: a["name"])

# Por múltiplos critérios: status, depois nome
agentes_ord = sorted(agentes, key=lambda a: (a["status"], a["name"]))

# Decrescente por data de último check-in
recentes = sorted(agentes, key=lambda a: a["lastKeepAlive"], reverse=True)
```

Filtragem e agrupamento

```
# Agrupar agentes por sistema operacional
from collections import defaultdict

por_os: dict[str, list[dict]] = defaultdict(list)
for a in agentes:
    por_os[a["os"]["platform"]].append(a)

# Contar ocorrências – ranking de versões vulneráveis
from collections import Counter

versoes = Counter(a["version"] for a in agentes)
versoes.most_common(5) # [('4.9.2', 310), ('4.8.1', 120), ...]
```

Exemplo real: transformar JSON de API

Cenário: a API de um gerenciador de endpoints retorna a lista de agentes; você precisa de um relatório por plataforma com apenas os desconectados há mais de 24h.

```

from collections import defaultdict
from datetime import datetime, timedelta, timezone

def relatorio_desconectados(resposta_api: dict) -> dict[str, list[str]]:
    """Recebe o JSON bruto e devolve {plataforma: [nomes...]}. """
    limite = datetime.now(timezone.utc) - timedelta(hours=24)
    agentes = resposta_api["data"]["affected_items"]

    relatorio: dict[str, list[str]] = defaultdict(list)
    for a in agentes:
        if a.get("status") != "disconnected":
            continue
        ultimo = datetime.fromisoformat(a["lastKeepAlive"])
        if ultimo < limite:
            plataforma = a.get("os", {}).get("platform", "desconhecido")
            relatorio[plataforma].append(a["name"])

    return {plat: sorted(nomes) for plat, nomes in relatorio.items()}

```

Repare no padrão que você repetirá mil vezes: **navegar no dict** → **filtrar** → **transformar** → **reagrupar**.

Exercício rápido

Dada a lista `vulns = [{"agent": "srv-01", "cve": "CVE-2026-1111", "severity": "critical"}, ...]`, produza um dict `{"critical": 12, "high": 40, ...}` contando vulnerabilidades por severidade.

Solução:

```

from collections import Counter
contagem = Counter(v["severity"] for v in vulns)
resultado = dict(contagem)

```

Módulo 5 — Strings, datas e JSON

Manipulação de strings

```
linha = "  srv-web-01;10.0.30.11;active  "

linha.strip()                # remove espaços das pontas
linha.strip().split(";")     # ['srv-web-01', '10.0.30.11', 'active']
"srv-web-01".startswith("srv-") # True
"SRV-WEB-01".lower()
"web" in linha               # busca de substring
"-".join(["srv", "web", "01"]) # 'srv-web-01'
"srv-web-01".replace("web", "app")
"CVE-2026-1234".removeprefix("CVE-") # '2026-1234' (3.9+)
```

Multilinha e templates de mensagens (ex.: notificação de cobrança):

```
mensagem = (
    f"Olá {cliente['nome']},\n"
    f"Sua fatura de R$ {valor:.2f} vence em {vencimento:%d/%m/%Y}.\n"
    f"Linha digitável: {linha_digitavel}"
)
```

regex essencial

Não decore regex — decore estes cinco padrões:

```
import re

# 1. Extrair um valor
m = re.search(r"CVE-\d{4}-\d{4,7}", texto)
if m:
    cve = m.group(0)

# 2. Extrair todos
cves = re.findall(r"CVE-\d{4}-\d{4,7}", relatorio)

# 3. Grupos nomeados — parsear linha de log
padrao = re.compile(
    r"(?P<data>\d{4}-\d{2}-\d{2}) (?P<hora>\d{2}:\d{2}:\d{2}) "
    r"(?P<nivel>\w+) (?P<msg>.*)"
)
m = padrao.match("2026-07-29 14:03:22 ERROR conexão recusada")
if m:
    print(m.group("nivel"), "->", m.group("msg"))

# 4. Substituir (mascarar dados sensíveis em log)
seguro = re.sub(r"\d{3}\.\d{3}\.\d{3}-\d{2}", "****.***.***-**", texto)

# 5. Validar formato
if re.fullmatch(r"\d{1,3}(\.\d{1,3}){3}", ip):
    ...
```

Use `re.compile()` quando o padrão for aplicado em loop — evita recompilar a cada linha.

datetime e timezone

Regra profissional: armazene e transporte tudo em UTC; converta para o fuso local só na exibição.

```
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo

agora_utc = datetime.now(timezone.utc)
sp = ZoneInfo("America/Bahia")
agora_local = agora_utc.astimezone(sp)

# Aritmética
vencimento = agora_utc + timedelta(days=30)
atrasado = agora_utc > vencimento

# Parse de string ISO 8601 (padrão de APIs)
d = datetime.fromisoformat("2026-07-29T17:03:22+00:00")

# Parse de formato customizado
d = datetime.strptime("29/07/2026 14:30", "%d/%m/%Y %H:%M")

# Formatação
f"{agora_local:%d/%m/%Y %H:%M}"      # 29/07/2026 14:03
agora_utc.isoformat()                # para gravar/enviar
```

Cuidado clássico: `datetime.now()` sem timezone cria um datetime "naive". Comparar naive com aware lança `TypeError`. Sempre passe `timezone.utc`.

json

```
import json

# str/bytes -> objeto Python
dados = json.loads('{"id": 17, "ativo": true, "obs": null}')
# {'id': 17, 'ativo': True, 'obs': None}

# objeto Python -> str
corpo = json.dumps({"cliente_id": 42, "valor": 199.90}, ensure_ascii=False)

# Arquivos
with open("config.json", encoding="utf-8") as f:
    config = json.load(f)

with open("saida.json", "w", encoding="utf-8") as f:
    json.dump(relatorio, f, ensure_ascii=False, indent=2)
```

Mapeamento JSON ↔ Python:

JSON	Python
object	dict
array	list
string	str
number	int / float
true / false	True / False
null	None

`datetime` **não é serializável** por padrão. Solução rápida:

```
json.dumps(dados, default=str)    # converte o que não souber via str()
```

(Com Pydantic — Módulo 13 — isso é resolvido automaticamente.)

Exemplo real: processar resposta de API e converter datas

```
import json
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

LOCAL = ZoneInfo("America/Bahia")

def formatar_faturas(corpo_resposta: str) -> list[dict]:
    """Converte resposta JSON da API de pagamentos em registros prontos
    para exibição: datas locais e valores em reais formatados."""
    dados = json.loads(corpo_resposta)
    saida = []
    for fatura in dados["items"]:
        criada = datetime.fromisoformat(fatura["created_at"])
        vence = datetime.fromisoformat(fatura["due_date"])
        saida.append({
            "id": fatura["id"],
            "cliente": fatura["customer"]["name"],
            "valor": f"R$ {fatura['amount'] / 100:.2f}", # API envia centavos
            "criada_em": criada.astimezone(LOCAL).strftime("%d/%m/%Y %H:%M"),
            "vencimento": vence.astimezone(LOCAL).strftime("%d/%m/%Y"),
            "atrasada": vence < datetime.now(timezone.utc),
        })
    return saida
```

Exercício rápido

Escreva `extrair_ips(log: str) -> set[str]` que retorne todos os IPv4 únicos de um texto de log.
 Bônus: exclua IPs de rede privada `10.x.x.x`.

Módulo 6 — Arquivos e automação

Leitura e escrita

```
# SEMPRE use "with" – fecha o arquivo mesmo com exceção
with open("relatorio.txt", "w", encoding="utf-8") as f:
    f.write("linha 1\n")

with open("acesso.log", encoding="utf-8") as f:
    for linha in f:
        # itera linha a linha SEM carregar tudo
        if "ERROR" in linha:
            processar(linha)
```

Modos: `"r"` leitura (default), `"w"` sobrescreve, `"a"` acrescenta, `"rb"/"wb"` binário (imagens, PDFs, comprovantes).

pathlib — o jeito moderno de lidar com caminhos

Esqueça `os.path`. Use `Path`:

```
from pathlib import Path

base = Path("/var/dados/importacao")
arquivo = base / "clientes" / "2026-07.csv"    # / concatena caminhos

arquivo.exists()
arquivo.suffix          # '.csv'
arquivo.stem            # '2026-07'
arquivo.parent.mkdir(parents=True, exist_ok=True)

# Ler/escrever pequeno conteúdo em 1 linha
texto = arquivo.read_text(encoding="utf-8")
Path("token.txt").write_text(token)

# Varredura – todos os CSVs, recursivamente
for csv_path in base.rglob("*.csv"):
    print(csv_path, csv_path.stat().st_size, "bytes")
```

CSV

```
import csv
from pathlib import Path

# Leitura com DictReader: cada linha vira dict com as chaves do cabeçalho
with open("clientes.csv", encoding="utf-8", newline="") as f:
    for linha in csv.DictReader(f, delimiter=";"):
        print(linha["nome"], linha["cpf_cnpj"])

# Escrita
campos = ["agente", "cve", "severidade"]
with open("vulns.csv", "w", encoding="utf-8", newline="") as f:
    w = csv.DictWriter(f, fieldnames=campos, delimiter=";")
    w.writeheader()
    w.writerows(lista_de_dicts)
```

Arquivo gerado no Excel brasileiro? Use `delimiter=";"` e `encoding="utf-8-sig"` (ou `latin-1` para legados).

ZIP

```
import zipfile
from pathlib import Path

# Compactar backups/comprovantes do mês
with zipfile.ZipFile("comprovantes-2026-07.zip", "w", zipfile.ZIP_DEFLATED) as z:
    for pdf in Path("comprovantes").glob("2026-07*.pdf"):
        z.write(pdf, arcname=pdf.name)

# Extrair
with zipfile.ZipFile("export.zip") as z:
    z.extractall("dados_extraidos")
```

Variáveis de ambiente (.env)

Credenciais **nunca** vão para o código nem para o Git.

```
# .env (adicione ao .gitignore!)
API_BASE_URL=https://api.pagamentos.com/v1
API_TOKEN=sk_live_9f8a7b...
DATABASE_URL=postgresql://app:senha@localhost:5432/cobrancas
```

```
import os
from dotenv import load_dotenv    # pip install python-dotenv

load_dotenv()                    # lê o .env para o ambiente

API_TOKEN = os.environ["API_TOKEN"]    # KeyError se faltar (bom: falha cedo)
DEBUG = os.getenv("DEBUG", "false").lower() == "true"    # com default
```

Em Docker/produção, as variáveis vêm do orquestrador e o `load_dotenv()` simplesmente não encontra arquivo — mesmo código funciona nos dois ambientes.

Projeto: importar clientes/boletos de um CSV

Cenário real: a associação de farmácias envia um CSV mensal com títulos a receber. Valide, normalize e gere um JSON pronto para a API.

```
"""importador.py – CSV de títulos -> JSON validado."""
import csv
import json
import logging
import re
from datetime import datetime
from pathlib import Path

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
logger = logging.getLogger(__name__)

RE_DOC = re.compile(r"^\d{11}$|^d{14}$")  # CPF ou CNPJ, só dígitos

def limpar_documento(doc: str) -> str:
    return re.sub(r"\D", "", doc)

def parse_valor(valor: str) -> float:
    """'1.234,56' -> 1234.56"""
    return float(valor.replace(".", "").replace(",", "."))

def importar(caminho: Path) -> list[dict]:
    titulos, erros = [], 0
    with caminho.open(encoding="utf-8-sig", newline="") as f:
        for num, linha in enumerate(csv.DictReader(f, delimiter=";"), start=2):
            try:
                doc = limpar_documento(linha["cpf_cnpj"])
                if not RE_DOC.fullmatch(doc):
                    raise ValueError(f"documento inválido: {linha['cpf_cnpj']}")
                titulos.append({
                    "cliente": linha["nome"].strip().title(),
                    "documento": doc,
                    "valor": parse_valor(linha["valor"]),
                    "vencimento": datetime.strptime(
                        linha["vencimento"], "%d/%m/%Y"
                    ).date().isoformat(),
                })
            except (KeyError, ValueError) as e:
                erros += 1
                logger.warning("Linha %d ignorada: %s", num, e)
    logger.info("Importados %d títulos (%d erros)", len(titulos), erros)
    return titulos

if __name__ == "__main__":
    dados = importar(Path("titulos-2026-07.csv"))
    Path("titulos.json").write_text(
        json.dumps(dados, ensure_ascii=False, indent=2), encoding="utf-8"
    )
```

Esse script já demonstra o esqueleto de toda automação profissional: **ler → validar → transformar → registrar erros sem parar o lote → exportar**.

Módulo 7 — Tratamento de erros profissional

try / except

```
try:
    resposta = httpx.get(url, timeout=10)
    resposta.raise_for_status()
    dados = resposta.json()
except httpx.TimeoutException:
    logger.error("Timeout ao chamar %s", url)
except httpx.HTTPStatusError as e:
    logger.error("API retornou %s: %s", e.response.status_code, e.response.text)
```

Exceções específicas — sempre

```
# ERRADO: esconde bugs, captura até Ctrl+C indireto
try:
    processar(payload)
except Exception:
    pass

# CERTO: captura só o que você sabe tratar
try:
    valor = float(payload["valor"])
except KeyError:
    raise ValueError("campo 'valor' ausente no payload")
except (TypeError, ValueError):
    raise ValueError(f"valor não numérico: {payload.get('valor')}")
```

Hierarquia útil de lembrar: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `FileNotFoundError`, `PermissionError`, `TimeoutError`, `ConnectionError`, `json.JSONDecodeError` (herda de `ValueError`).

raise e exceções próprias

Crie exceções de domínio para o seu sistema — elas viram respostas HTTP limpas no FastAPI:

```
class ErroIntegracao(Exception):
    """Falha ao comunicar com serviço externo."""

class TituloDuplicadoError(Exception):
    def __init__(self, documento: str, vencimento: str):
        super().__init__(f"Título já existe: {documento} venc. {vencimento}")
        self.documento = documento

# Re-lançar preservando a causa original (traceback encadeado)
try:
    r = httpx.post(url, json=payload, timeout=10)
    r.raise_for_status()
except httpx.HTTPError as e:
    raise ErroIntegracao(f"Gateway de pagamento indisponível") from e
```

finally e else

```
conexao = abrir_conexao()
try:
    executar(conexao)
except OperationalError:
    logger.exception("Falha no banco")
    raise
else:
    logger.info("Executado com sucesso")  # roda só se NÃO houve exceção
finally:
    conexao.close()  # roda SEMPRE
```

(Na prática, prefira context managers — `with` — que fazem o `finally` por você.)

logging básico (aprofundado no Módulo 18)

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
logger = logging.getLogger(__name__)

logger.info("Sincronizando %d agentes", len(agentes))
logger.warning("Agente %s sem keep-alive há 48h", nome)

try:
    sincronizar()
except ErroIntegracao:
    logger.exception("Sincronização falhou")  # loga com traceback completo
```

`logger.exception()` dentro de um `except` é a forma correta de registrar o stack trace. **Nunca** use `print()` para erro em produção.

Os quatro erros que você vai tratar toda semana

```
import json
import httpx
from pathlib import Path

def buscar_config(url: str, fallback: Path) -> dict:
    # 1) Erro de rede 2) Timeout
    try:
        r = httpx.get(url, timeout=5)
        r.raise_for_status()
    except httpx.TimeoutException:
        logger.warning("Timeout na API de config; usando fallback local")
    except httpx.ConnectError:
        logger.warning("API inacessível; usando fallback local")
    else:
        # 3) JSON inválido
        try:
            return r.json()
        except json.JSONDecodeError:
            logger.error("Resposta não é JSON válido: %.200s", r.text)

    # 4) Arquivo inexistente
    try:
        return json.loads(fallback.read_text(encoding="utf-8"))
    except FileNotFoundError:
        raise RuntimeError(f"Sem API e sem fallback ({fallback}). Abortando.")
```

Exercício rápido

Escreva `ler_json_seguro(caminho: str) -> dict | None` que retorne o conteúdo do arquivo JSON, ou `None` se o arquivo não existir ou for inválido — logando um warning distinto para cada caso.

Módulo 8 — Orientação a objetos sem complicação

Em backend Python moderno, classes servem principalmente para: **modelar dados** (dataclass/Pydantic), **agrupar operações com estado compartilhado** (clientes de API, repositórios) e **definir contratos** (protocolos). Nada de hierarquias profundas.

Classe básica e init

```
class ClienteApiPagamentos:
    """Encapsula autenticação e chamadas ao gateway."""

    def __init__(self, base_url: str, token: str, timeout: int = 10):
        self.base_url = base_url.rstrip("/")
        self.timeout = timeout
        self._headers = {"Authorization": f"Bearer {token}"} # _ = "privado"

    def criar_cobranca(self, documento: str, valor_centavos: int) -> dict:
        r = httpx.post(
            f"{self.base_url}/charges",
            json={"document": documento, "amount": valor_centavos},
            headers=self._headers,
            timeout=self.timeout,
        )
        r.raise_for_status()
        return r.json()

api = ClienteApiPagamentos("https://api.pag.com/v1", token=os.environ["TOKEN"])
cobranca = api.criar_cobranca("12345678901", 19990)
```

Por que classe aqui? Porque `base_url`, `token` e `timeout` seriam repetidos em toda função. A classe carrega o estado; os métodos usam.

dataclass — modelagem de dados sem boilerplate

```
from dataclasses import dataclass, field
from datetime import date

@dataclass
class Titulo:
    documento: str
    valor: float
    vencimento: date
    pago: bool = False
    tags: list[str] = field(default_factory=list) # default mutável seguro

t = Titulo("12345678901", 199.90, date(2026, 8, 10))
print(t) # __repr__ automático e legível
t == Titulo(...) # __eq__ automático (compara campos)
```

`@dataclass` gera `__init__`, `__repr__` e `__eq__`. Para dados que atravessam a fronteira do sistema (API, arquivo), você usará Pydantic (validação); para dados internos, dataclass basta.

Propriedades

Atributo calculado com cara de atributo:

```
from datetime import date

@dataclass
class Titulo:
    valor: float
    vencimento: date

    @property
    def atrasado(self) -> bool:
        return date.today() > self.vencimento

    @property
    def valor_com_multa(self) -> float:
        return round(self.valor * 1.02, 2) if self.atrasado else self.valor

t.atrasado           # sem parênteses – parece atributo, executa código
```

Composição > herança

Monte objetos a partir de outros objetos:

```
@dataclass
class Cliente:
    nome: str
    documento: str
    whatsapp: str

@dataclass
class Pedido:
    cliente: Cliente           # composição
    itens: list[str]
    total: float

@dataclass
class Pagamento:
    pedido: Pedido             # composição
    metodo: str                # "pix" | "boleto" | "cartao"
    confirmado: bool = False

    def confirmar(self) -> None:
        self.confirmado = True
        logger.info("Pagamento %s de %s confirmado",
                    self.metodo, self.pedido.cliente.nome)
```

Herança — quando evitar

Use herança **apenas** para: exceções próprias, classes base de framework (`BaseModel` do Pydantic, `DeclarativeBase` do SQLAlchemy) e casos raros de especialização real. Evite quando:

- A relação é "tem um" e não "é um" → composição.
- Você herdaria só para reaproveitar 2 métodos → funções utilitárias.
- Criaria mais de 2 níveis de hierarquia → repense o design.

Protocolos — contratos sem herança

```
from typing import Protocol

class Notificador(Protocol):
    def enviar(self, destino: str, mensagem: str) -> None: ...

class NotificadorWhatsApp:
    def enviar(self, destino: str, mensagem: str) -> None:
        httpx.post(WA_URL, json={"to": destino, "text": mensagem})

class NotificadorEmail:
    def enviar(self, destino: str, mensagem: str) -> None:
        ...

def cobrar(titulo: Titulo, notificador: Notificador) -> None:
    notificador.enviar(titulo.cliente.whatsapp, f"Fatura vence dia ...")
```

`NotificadorWhatsApp` não herda de nada — basta ter o método com a assinatura certa (duck typing verificável pelo Pylance). É assim que se faz "interface" em Python moderno.

Exercício rápido

Modele `Cliente`, `ContaReceber` (valor, vencimento, cliente) e `Recebimento` (conta, data, valor_pago) com dataclasses. Adicione em `ContaReceber` a property `saldo_devedor` que subtrai os recebimentos.

Módulo 9 — Tipagem moderna (MUITO IMPORTANTE)

Type hints não mudam a execução — mas alimentam o Pylance (erros antes de rodar), o FastAPI (validação e documentação automáticas) e o Pydantic (parsing). **Em 2026, código backend sem tipos é considerado legado.**

Sintaxe essencial (Python 3.12)

```
def buscar_agente(agent_id: str) -> dict:
    ...

nome: str = "srv-web-01"
portas: list[int] = [1514, 1515, 55000]
config: dict[str, str] = {"host": "10.0.0.1"}
par: tuple[str, int] = ("srv-web-01", 1514)
tags: set[str] = {"prod", "web"}
```

Optional e Union — o novo jeito

```
# Moderno (3.10+): use o operador |
def achar_cliente(doc: str) -> Cliente | None:      # pode não achar
    ...

def parse_porta(valor: str | int) -> int:           # aceita ambos
    return int(valor)

# Forma antiga (você verá em código legado; evite em código novo):
# Optional[Cliente] == Cliente | None
# Union[str, int]   == str | int
```

O ganho imediato: o Pylance te força a tratar o `None`:

```
cliente = achar_cliente("123")
print(cliente.nome)      # Pylance: "nome" is not a known attribute of "None"

if cliente is not None:
    print(cliente.nome)   # ok – tipo "estreitado" para Cliente
```

dict[str, Any] e o custo dele

```
from typing import Any

def processar_webhook(payload: dict[str, Any]) -> None: ...
```

`Any` desliga a checagem — use só na fronteira de entrada, e converta para tipos ricos o quanto antes (TypedDict ou Pydantic).

TypedDict — dar forma a JSON

```
from typing import TypedDict, NotRequired

class AgenteDict(TypedDict):
    id: str
    name: str
    status: str
    ip: NotRequired[str]      # chave opcional

def desconectados(agentes: list[AgenteDict]) -> list[str]:
    return [a["name"] for a in agentes if a["status"] == "disconnected"]
    # a["nme"] -> Pylance acusa erro de digitação em tempo de edição!
```

TypedDict é ideal quando o dado **já é** dict (resposta `.json()`) e você quer autocomplete + verificação sem instanciar objetos.

Literal — valores permitidos

```
from typing import Literal

Severidade = Literal["low", "medium", "high", "critical"]

def filtrar_por_severidade(vulns: list[dict], sev: Severidade) -> list[dict]:
    ...

filtrar_por_severidade(vulns, "critical")    #
filtrar_por_severidade(vulns, "urgent")     # erro no editor, antes de rodar
```

Aliases de tipo e genéricos rápidos

```
type JsonDict = dict[str, Any]                # 3.12: alias explícito
type Resultado[T] = tuple[bool, T | None, str]

def tentar[T](fn: Callable[[], T]) -> Resultado[T]:
    try:
        return True, fn(), ""
    except Exception as e:
        return False, None, str(e)
```

Por que isso é essencial para FastAPI e Pydantic

No FastAPI, o **type hint** **É a especificação**:

```
@app.get("/agentes/{agent_id}")
def obter_agente(agent_id: int, incluir_vulns: bool = False) -> AgenteOut:
    ...
```

Desta assinatura o framework extrai, sozinho: conversão de `agent_id` para int (404/422 automático se inválido), parsing do query param `incluir_vulns`, validação da resposta contra `AgenteOut` e a documentação Swagger completa. Você escreve tipos; o framework escreve o resto. Sem hints, o FastAPI simplesmente não funciona direito — por isso este módulo vem **antes** dele.

Exercício rápido

Tipifique a função abaixo (parâmetros e retorno) usando `TypedDict` para o item e `Literal` para o status:

```
def resumir(titulos, status):  
    return [t["documento"] for t in titulos if t["status"] == status]
```

Módulo 10 — HTTP do zero ao profissional

requests — o clássico

```
import requests

BASE = "https://api.exemplo.com/v1"
HEADERS = {"Authorization": f"Bearer {TOKEN}", "Accept": "application/json"}

# GET com query params
r = requests.get(f"{BASE}/agents",
                 params={"status": "active", "limit": 100},
                 headers=HEADERS, timeout=10)
r.status_code          # 200
r.headers["content-type"]
dados = r.json()        # parse do corpo JSON

# POST com corpo JSON
r = requests.post(f"{BASE}/charges",
                  json={"document": "12345678901", "amount": 19990},
                  headers=HEADERS, timeout=10)

# PUT (substituir) e PATCH (alterar parcialmente)
requests.put(f"{BASE}/agents/017", json=agente_completo, headers=HEADERS, timeout=10)
requests.patch(f"{BASE}/agents/017", json={"group": "web"}, headers=HEADERS, timeout=10)

# DELETE
requests.delete(f"{BASE}/agents/017", headers=HEADERS, timeout=10)
```

Timeout: NUNCA omitta

Sem `timeout`, uma API travada congela seu processo **para sempre**. Padrão profissional:

```
r = requests.get(url, timeout=(3.05, 27)) # (conexão, leitura)
```

Verifique o status sempre

```
r = requests.get(url, timeout=10)
r.raise_for_status() # lança requests.HTTPError para 4xx/5xx
dados = r.json()     # só chega aqui se 2xx
```

Sessões — conexão persistente + config compartilhada

```
sessao = requests.Session()
sessao.headers.update({"Authorization": f"Bearer {TOKEN}"})

for pagina in range(1, 51):
    r = sessao.get(f"{BASE}/vulnerabilities",
                  params={"page": pagina}, timeout=10)
    processar(r.json())
```

`Session` reaproveita a conexão TCP/TLS (keep-alive): em 50 requisições, a diferença de tempo é brutal. **Toda integração séria usa Session.**

Upload e download de arquivos

```
# Upload multipart (ex.: comprovante de pagamento)
with open("comprovante.pdf", "rb") as f:
    r = sessao.post(f"{BASE}/receipts",
                    files={"file": ("comprovante.pdf", f, "application/pdf")},
                    data={"titulo_id": "873"}, timeout=30)

# Download em streaming (não carrega o arquivo inteiro na RAM)
with sessao.get(f"{BASE}/reports/2026-07.pdf", stream=True, timeout=60) as r:
    r.raise_for_status()
    with open("relatorio.pdf", "wb") as f:
        for chunk in r.iter_content(chunk_size=65536):
            f.write(chunk)
```

HTTPX — o moderno

Mesma interface do requests, com três vantagens: **HTTP/2**, **cliente assíncrono** e **timeout obrigatório por padrão** (5s se você esquecer — falha segura).

Cliente síncrono

```
import httpx

with httpx.Client(base_url="https://api.exemplo.com/v1",
                  headers={"Authorization": f"Bearer {TOKEN}"},
                  timeout=10.0) as client:
    r = client.get("/agents", params={"status": "active"})
    r.raise_for_status()
    agentes = r.json()["data"]
```

Note o `base_url`: os métodos recebem só o caminho. Config num lugar só.

Cliente assíncrono

```
import asyncio
import httpx

async def buscar_status(client: httpx.AsyncClient, agent_id: str) -> dict:
    r = await client.get(f"/agents/{agent_id}")
    r.raise_for_status()
    return r.json()

async def main() -> None:
    async with httpx.AsyncClient(base_url=BASE, headers=HEADERS,
                                timeout=10.0) as client:
        tarefas = [buscar_status(client, i) for i in ids]
        resultados = await asyncio.gather(*tarefas) # 100 chamadas em paralelo

asyncio.run(main())
```

requests vs httpx — quando usar cada um

Critério	requests	httpx
Scripts e automações simples	perfeito	ok
Async (muitas chamadas em paralelo)	não tem	
Dentro de rotas async do FastAPI	bloqueia o event loop	obrigatório
HTTP/2		
Timeout default	não tem (perigo)	5s
Maturidade/exemplos na internet	enorme	grande e crescendo

Regra prática 2026: projeto novo → httpx. Script rápido ou codebase legada → requests continua excelente.

Exercícios

1. Consumir API pública:

```
import httpx

r = httpx.get("https://api.github.com/repos/wazuh/wazuh", timeout=10)
r.raise_for_status()
repo = r.json()
print(f"{repo['full_name']}: {repo['stargazers_count']} stars, "
      f"issue abertas: {repo['open_issues_count']}")
```

2. Autenticar com Bearer Token e enviar JSON:

```
with httpx.Client(base_url="https://httpbin.org", timeout=10) as c:
    r = c.post("/anything",
               headers={"Authorization": "Bearer teste-123"},
               json={"evento": "curso", "modulo": 10})
    corpo = r.json()
    assert corpo["headers"]["Authorization"] == "Bearer teste-123"
    assert corpo["json"]["modulo"] == 10
```

3. Baixar arquivo com barra de progresso simples:

```
url = "https://speed.hetzner.de/1MB.bin"
with httpx.stream("GET", url, timeout=60) as r:
    total = int(r.headers.get("content-length", 0))
    baixado = 0
    with open("teste.bin", "wb") as f:
        for chunk in r.iter_bytes(65536):
            f.write(chunk)
            baixado += len(chunk)
        print(f"\r{baixado * 100 // total}% ", end="")
```

Módulo 11 — APIs REST na prática

Os conceitos em uma página

REST organiza a API em **recursos** (substantivos, no plural) manipulados por **métodos HTTP** (verbos):

```
GET    /clientes          lista clientes
POST   /clientes          cria cliente
GET    /clientes/42      detalhe do cliente 42
PUT    /clientes/42      substitui o cliente 42
PATCH /clientes/42      altera campos do cliente 42
DELETE /clientes/42      remove o cliente 42
GET    /clientes/42/titulos titulos do cliente 42 (recurso aninhado)
```

Errado (verbos na URL): `POST /criarCliente`, `GET /buscarTitulosDoCliente`.

Status codes que você realmente usa

Código	Significado	Quando usar/esperar
200 OK	Sucesso genérico	GET, PUT, PATCH bem-sucedidos
201 Created	Recurso criado	Resposta de POST
204 No Content	Sucesso sem corpo	DELETE bem-sucedido
301/308 Redirect	URL mudou	Siga o header <code>Location</code>
400 Bad Request	Requisição malformada	JSON inválido, parâmetro errado
401 Unauthorized	Não autenticado	Token ausente/expirado → renove
403 Forbidden	Autenticado sem permissão	Não adianta renovar token
404 Not Found	Recurso não existe	ID errado ou rota errada
409 Conflict	Conflito de estado	Registro duplicado
422 Unprocessable	Validação falhou	Corpo bem-formado, dados inválidos (padrão FastAPI)
429 Too Many Requests	Rate limit	Espere o header <code>Retry-After</code>
500 Internal Error	Bug no servidor	Retry pode ajudar
502/503/504	Gateway/indisponível	Retry com backoff

Memorize a leitura por família: **2xx sucesso**, **3xx redirecionamento**, **4xx erro seu**, **5xx erro deles**.

Idempotência

Uma operação é idempotente se repeti-la produz o mesmo estado final. **Importa porque redes falham e retries acontecem.**

Método	Idempotente?	Consequência
GET, PUT, DELETE		Pode dar retry sem medo
POST		Retry pode criar cobrança duplicada!

Solução usada por gateways de pagamento: header `Idempotency-Key` com um UUID gerado por você — o servidor ignora repetições da mesma chave:

```
import uuid
client.post("/charges", json=payload,
            headers={"Idempotency-Key": str(uuid.uuid4())})
```

Paginação

Nenhuma API séria devolve 500 mil registros de uma vez. Dois estilos:

```
# 1) offset/limit (ou page/per_page)
def listar_tudo(client: httpx.Client, rota: str) -> list[dict]:
    itens, offset, limite = [], 0, 500
    while True:
        r = client.get(rota, params={"offset": offset, "limit": limite})
        r.raise_for_status()
        lote = r.json()["data"]["affected_items"]
        itens.extend(lote)
        if len(lote) < limite:           # última página
            return itens
        offset += limite

# 2) cursor (mais moderno; a resposta traz o ponteiro da próxima página)
cursor = None
while True:
    params = {"cursor": cursor} if cursor else {}
    corpo = client.get(rota, params=params).json()
    itens.extend(corpo["items"])
    cursor = corpo.get("next_cursor")
    if not cursor:
        break
```

Filtros, ordenação e versionamento

```
GET /titulos?status=vencido&min_valor=100&sort=-vencimento
GET /v1/clientes      ← versão na URL (mais comum)
```

Versione desde o dia 1 (`/v1/`): quando precisar quebrar contrato, você cria `/v2/` sem derrubar integrações existentes.

OpenAPI / Swagger

OpenAPI é a especificação (um JSON/YAML descrevendo cada rota, parâmetro e schema); Swagger UI é a interface gráfica que renderiza isso como documentação interativa com botão "Try it out". No FastAPI você ganha ambos automaticamente em `/docs` e `/openapi.json` — e times consomem esse JSON para gerar clientes tipados automaticamente.

Exercício rápido

Sem olhar a tabela: qual status para (a) token expirado, (b) criar cliente com CPF já cadastrado, (c) payload com data em formato errado, (d) exclusão bem-sucedida? (Respostas: 401, 409, 400/422, 204.)

Módulo 12 — FastAPI (núcleo do curso)

Instalação e primeiro endpoint

```
pip install "fastapi[standard]"
```

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="API de Tarefas", version="1.0.0")

@app.get("/health")
def healthcheck() -> dict:
    return {"status": "ok"}
```

```
fastapi dev main.py          # dev server com reload — http://127.0.0.1:8000
# produção: uvicorn main:app --host 0.0.0.0 --port 8000
```

Abra **http://127.0.0.1:8000/docs** — o Swagger já está lá.

Path params, query params e body

O FastAPI decide pela assinatura da função: **está na rota** → **path param**; **tipo simples fora da rota** → **query param**; **BaseModel** → **body JSON**.

```
from pydantic import BaseModel, Field

class TarefaIn(BaseModel):
    titulo: str = Field(min_length=3, max_length=120)
    descricao: str | None = None
    prioridade: int = Field(default=3, ge=1, le=5)

@app.get("/tarefas/{tarefa_id}")          # path param
def obter(tarefa_id: int): ...

@app.get("/tarefas")                     # query params
def listar(status: str = "pendente", limit: int = 20, offset: int = 0): ...

@app.post("/tarefas", status_code=201)    # body
def criar(tarefa: TarefaIn): ...
```

Envie `{"titulo": "ab"}` e receba um **422 automático** com a explicação exata do erro — sem uma linha de validação manual.

Response model

Separe o schema de entrada do de saída (a saída tem `id`, datas, e **nunca** vaza campos sensíveis):

```

from datetime import datetime

class TarefaOut(BaseModel):
    id: int
    titulo: str
    prioridade: int
    criada_em: datetime

@app.post("/tarefas", response_model=TarefaOut, status_code=201)
def criar(tarefa: TarefaIn) -> TarefaOut:
    ...

```

`response_model` filtra a resposta: se o objeto interno tiver `senha_hash`, ele não sai.

Erros HTTP

```

from fastapi import HTTPException

@app.get("/tarefas/{tarefa_id}", response_model=TarefaOut)
def obter(tarefa_id: int):
    tarefa = repositario.buscar(tarefa_id)
    if tarefa is None:
        raise HTTPException(status_code=404, detail="Tarefa não encontrada")
    return tarefa

```

Organização por routers

Nada de API inteira num arquivo. Estrutura profissional:

```

app/
├── main.py           # cria o app, inclui routers, middleware
├── core/
│   ├── config.py     # settings via env vars
│   └── security.py    # JWT, hash de senha
├── routers/
│   ├── tarefas.py
│   └── auth.py
├── schemas/          # modelos Pydantic (entrada/saída)
│   └── tarefa.py
├── services/         # regra de negócio
│   └── tarefa_service.py
└── repositories/     # acesso a dados (SQLAlchemy)
    └── tarefa_repo.py

```

```
# app/routers/tarefas.py
from fastapi import APIRouter

router = APIRouter(prefix="/tarefas", tags=["tarefas"])

@router.get("", response_model=list[TarefaOut])
def listar(): ...

# app/main.py
from fastapi import FastAPI
from app.routers import tarefas, auth

app = FastAPI(title="API de Tarefas")
app.include_router(tarefas.router)
app.include_router(auth.router)
```

O fluxo de uma requisição: **router** (HTTP) → **service** (negócio) → **repository** (dados). Router não conhece SQL; repository não conhece HTTP. Isso torna cada camada testável isoladamente.

Dependency Injection

`Depends()` é o mecanismo central do FastAPI — injeta recursos (sessão de banco, usuário autenticado, paginação) em qualquer rota:

```
from typing import Annotated
from fastapi import Depends
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db          # entrega a sessão à rota
    finally:
        db.close()       # fecha SEMPRE, mesmo com exceção

DbDep = Annotated[Session, Depends(get_db)]

@router.get("", response_model=list[TarefaOut])
def listar(db: DbDep, limit: int = 20):
    return db.scalars(select(Tarefa).limit(limit)).all()
```

Dependências podem depender de outras dependências (ex.: `get_current_user` depende de `get_db` + token). O FastAPI resolve a árvore toda por requisição — e nos testes você substitui qualquer dependência com `app.dependency_overrides`.

Middleware e CORS

```
import time
from fastapi import Request
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://painel.suaempresa.com.br"], # nunca "*" com credenciais
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.middleware("http")
async def log_tempo(request: Request, call_next):
    inicio = time.perf_counter()
    resposta = await call_next(request)
    ms = (time.perf_counter() - inicio) * 1000
    resposta.headers["X-Process-Time-Ms"] = f"{ms:.1f}"
    return resposta
```

CORS só importa quando um **navegador** em outro domínio chama sua API. Integrações servidor-a-servidor ignoram CORS.

Upload de arquivos

```
from fastapi import UploadFile, HTTPException
from pathlib import Path

DESTINO = Path("uploads")

@router.post("/tarefas/{tarefa_id}/anexo", status_code=201)
async def anexar(tarefa_id: int, arquivo: UploadFile):
    if arquivo.content_type != "application/pdf":
        raise HTTPException(415, "Apenas PDF")
    conteudo = await arquivo.read()
    if len(conteudo) > 5 * 1024 * 1024:
        raise HTTPException(413, "Máximo 5 MB")
    destino = DESTINO / f"{tarefa_id}_{arquivo.filename}"
    destino.write_bytes(conteudo)
    return {"arquivo": destino.name, "bytes": len(conteudo)}
```

Background tasks

Responda rápido; processe depois (essencial para webhooks):

```
from fastapi import BackgroundTasks

@router.post("/tarefas", status_code=201)
def criar(tarefa: TarefaIn, bg: BackgroundTasks, db: DbDep):
    nova = service.criar(db, tarefa)
    bg.add_task(notificar_whatsapp, nova.id) # roda após a resposta ser enviada
    return nova
```

WebSockets (introdução)

```
from fastapi import WebSocket

@app.websocket("/ws/alertas")
async def alertas(ws: WebSocket):
    await ws.accept()
    while True:
        evento = await fila_alertas.get()
        await ws.send_json(evento)
```

Use para dashboards em tempo real; para o resto, REST resolve.

Projeto guiado: API de tarefas

Neste ponto você tem tudo para montar a API completa com a estrutura de pastas acima: CRUD de tarefas com validação, paginação (`limit/offset`), filtro por status, upload de anexo e healthcheck. Nos Módulos 14 e 16 vamos plugar PostgreSQL e JWT nela — não jogue o código fora. A versão final está no Módulo 23.

Módulo 13 — Pydantic v2

Pydantic é o motor de validação do FastAPI — mas você o usará também **sozinho**, para validar webhooks, arquivos, config e respostas de APIs externas.

BaseModel e Field

```
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, Field, EmailStr

class Cliente(BaseModel):
    nome: str = Field(min_length=3)
    documento: str = Field(pattern=r"^\d{11}$|^\d{14}$")
    email: EmailStr | None = None
    limite_credito: float = Field(default=0, ge=0)
    origem: Literal["site", "whatsapp", "balcao"] = "balcao"
    criado_em: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
```

```
c = Cliente(nome="Farmácia Boa Saúde", documento="12345678000199",
            limite_credito="1500.50")      # str -> float automático (coerção)

Cliente(nome="ab", documento="123")
# ValidationError: 2 validation errors ... (mensagens exatas, campo a campo)
```

Validações customizadas

```
from pydantic import field_validator, model_validator

class Titulo(BaseModel):
    valor: float = Field(gt=0)
    vencimento: date
    desconto: float = 0

    @field_validator("documento")
    @classmethod
    def validar_documento(cls, v: str) -> str:
        v = re.sub(r"\D", "", v)          # normaliza: remove máscara
        if len(v) not in (11, 14):
            raise ValueError("CPF/CNPJ inválido")
        return v

    @model_validator(mode="after")
    def desconto_menor_que_valor(self) -> "Titulo":
        if self.desconto >= self.valor:
            raise ValueError("desconto não pode ser >= valor")
        return self
```

Serialização: model_dump e model_dump_json

```
t = Titulo(valor=199.9, vencimento=date(2026, 8, 10))

t.model_dump()                # dict Python
t.model_dump(mode="json")     # dict serializável (date -> "2026-08-10")
t.model_dump_json()           # string JSON direto
t.model_dump(exclude={"desconto"}) # omitir campos
t.model_dump(exclude_none=True)  # omitir campos None
```

Parsing: model_validate e model_validate_json

```
# dict -> modelo (ex.: linha do banco, resposta de API)
t = Titulo.model_validate(dados_dict)

# bytes/str JSON -> modelo, em um passo (mais rápido que json.loads + validate)
t = Titulo.model_validate_json(corpo_bruto)
```

Aliases — quando o JSON externo usa outro padrão

APIs externas costumam usar camelCase; seu código, snake_case:

```
from pydantic import ConfigDict

class PagamentoWebhook(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    evento_id: str = Field(alias="eventId")
    valor_centavos: int = Field(alias="amountInCents")
    pago_em: datetime = Field(alias="paidAt")

p = PagamentoWebhook.model_validate({"eventId": "evt_1",
                                     "amountInCents": 19990,
                                     "paidAt": "2026-07-29T14:00:00Z"})

p.valor_centavos          # 19990 – código limpo em snake_case
p.model_dump(by_alias=True) # volta para camelCase se precisar reenviar
```

Exemplo real: payload de webhook de pagamento

```
from typing import Literal
from pydantic import BaseModel, Field, ValidationError

class DadosCobranca(BaseModel):
    id: str
    document: str
    amount: int = Field(gt=0) # centavos
    status: Literal["paid", "refused", "expired", "refunded"]
    paid_at: datetime | None = None

class EventoWebhook(BaseModel):
    event_id: str = Field(alias="eventId")
    type: Literal["charge.paid", "charge.refused", "charge.expired"]
    created_at: datetime = Field(alias="createdAt")
    data: DadosCobranca # modelo aninhado

def processar(corpo_bruto: bytes) -> None:
    try:
        evento = EventoWebhook.model_validate_json(corpo_bruto)
    except ValidationError as e:
        logger.warning("Webhook inválido: %s", e.errors())
        return
    if evento.type == "charge.paid":
        baixar_titulo(evento.data.document, evento.data.amount / 100)
```

Repare: o modelo aninhado (`data: DadosCobranca`) valida a estrutura inteira de uma vez. Um único `model_validate_json` substitui dezenas de `if "campo" in dados`.

Settings tipadas (bônus muito usado)

```
# pip install pydantic-settings
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    api_token: str
    debug: bool = False

    model_config = {"env_file": ".env"}

settings = Settings() # lê env vars + .env, valida tipos, falha cedo
```

Exercício rápido

Modele o webhook `{"messageId": "...", "from": "5571999998888", "text": "PAGO", "receivedAt": "..."}` com aliases, e escreva a função que retorna `True` se o texto (normalizado) for "pago", "ok" ou "confirmado".

Módulo 14 — Banco de dados com SQLAlchemy

SQLAlchemy 2.x é o ORM padrão do ecossistema. Começamos com SQLite (zero setup) e migramos para PostgreSQL trocando **uma linha**.

Engine e Session

```
# app/core/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

# Desenvolvimento:
DATABASE_URL = "sqlite:///./tarefas.db"
# Produção (mesmo código, outra URL):
# DATABASE_URL = "postgresql+psycopg://app:senha@localhost:5432/tarefas"

engine = create_engine(DATABASE_URL, echo=False) # echo=True mostra o SQL
SessionLocal = sessionmaker(bind=engine, autoflush=False)

class Base(DeclarativeBase):
    pass
```

- **Engine:** pool de conexões, um por aplicação.
- **Session:** unidade de trabalho (transação), uma por requisição.

Models (mapeamento 2.x com Mapped)

```
from datetime import datetime, timezone
from sqlalchemy import String, ForeignKey, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Cliente(Base):
    __tablename__ = "clientes"

    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(120))
    documento: Mapped[str] = mapped_column(String(14), unique=True, index=True)
    whatsapp: Mapped[str | None] # nullable inferido do tipo
    criado_em: Mapped[datetime] = mapped_column(server_default=func.now())

    titulos: Mapped[list["Titulo"]] = relationship(back_populates="cliente")

class Titulo(Base):
    __tablename__ = "titulos"

    id: Mapped[int] = mapped_column(primary_key=True)
    cliente_id: Mapped[int] = mapped_column(ForeignKey("clientes.id"))
    valor: Mapped[float]
    vencimento: Mapped[datetime]
    pago: Mapped[bool] = mapped_column(default=False)

    cliente: Mapped[Cliente] = relationship(back_populates="titulos")
```

Os `Mapped[...]` são type hints de verdade: Pylance autocompleta colunas e acusa erro de tipo em queries.

```
Base.metadata.create_all(engine)      # cria as tabelas (só p/ dev; prod usa Alembic)
```

CRUD com a sintaxe 2.x

```
from sqlalchemy import select

with SessionLocal() as db:
    # CREATE
    cliente = Cliente(nome="Farmácia Boa Saúde", documento="12345678000199")
    db.add(cliente)
    db.commit()
    db.refresh(cliente)          # carrega o id gerado

    # READ
    c = db.get(Cliente, 1)      # por PK
    c = db.scalar(select(Cliente).where(Cliente.documento == "123..."))
    vencidos = db.scalars(
        select(Titulo)
        .where(Titulo.pago == False, Titulo.vencimento < datetime.now(timezone.utc))
        .order_by(Titulo.vencimento)
        .limit(50)
    ).all()

    # UPDATE
    titulo = db.get(Titulo, 10)
    titulo.pago = True
    db.commit()

    # DELETE
    db.delete(titulo)
    db.commit()
```

Relacionamentos na prática

```
# Acessar do lado "um"
cliente.titulos # lista de Titulo (lazy load)

# JOIN explícito com carregamento antecipado (evita o problema N+1)
from sqlalchemy.orm import selectinload

clientes = db.scalars(
    select(Cliente).options(selectinload(Cliente.titulos))
).all()
for c in clientes:
    total = sum(t.valor for t in c.titulos if not t.pago) # sem query extra

# Agregação direto no banco
from sqlalchemy import func
resultado = db.execute(
    select(Cliente.nome, func.sum(Titulo.valor))
    .join(Titulo)
    .where(Titulo.pago == False)
    .group_by(Cliente.nome)
).all()
```

Armadilha N+1: iterar clientes e acessar `c.titulos` sem `selectinload` dispara uma query por cliente. Com 500 clientes = 501 queries. Sempre que for iterar relacionamentos, carregue antecipadamente.

Migrations com Alembic

Em produção você nunca usa `create_all` — usa migrations versionadas:

```
pip install alembic
alembic init migrations
```

Configure em `migrations/env.py`:

```
from app.core.database import Base
from app import models # importa TODOS os models
target_metadata = Base.metadata
```

E em `alembic.ini` (ou via env var): `sqlalchemy.url = postgresql+psycopg://...`

Fluxo de trabalho:

```
# Alterou um model? Gere a migration automaticamente:
alembic revision --autogenerate -m "adiciona coluna whatsapp"

# Revise o arquivo gerado (SEMPRE), depois aplique:
alembic upgrade head

# Reverter a última:
alembic downgrade -1
```

Cada migration é um arquivo Python versionado no Git — o histórico do schema acompanha o histórico do código, igual você já faz com infraestrutura como código.

SQLite → PostgreSQL

```
pip install "psycopg[binary]"
docker run -d --name pg -e POSTGRES_PASSWORD=senha -e POSTGRES_DB=tarefas \
-p 5432:5432 postgres:17
```

```
DATABASE_URL = "postgresql+psycopg://postgres:senha@localhost:5432/tarefas"
```

Todo o resto do código permanece idêntico. Diferenças que importam: PostgreSQL tem tipos ricos (JSONB, arrays, UUID nativo), concorrência real de escrita e é o que roda em produção. SQLite fica para dev e testes.

Projeto: persistir a API de tarefas

Integre com o FastAPI usando a dependência `get_db` do Módulo 12:

```
# app/repositories/tarefa_repo.py
from sqlalchemy import select
from sqlalchemy.orm import Session

class TarefaRepo:
    def __init__(self, db: Session):
        self.db = db

    def criar(self, dados: TarefaIn) -> Tarefa:
        tarefa = Tarefa(**dados.model_dump())
        self.db.add(tarefa)
        self.db.commit()
        self.db.refresh(tarefa)
        return tarefa

    def listar(self, status: str | None, limit: int, offset: int) -> list[Tarefa]:
        stmt = select(Tarefa).limit(limit).offset(offset)
        if status:
            stmt = stmt.where(Tarefa.status == status)
        return list(self.db.scalars(stmt))

# app/routers/tarefas.py
@router.post("", response_model=TarefaOut, status_code=201)
def criar(dados: TarefaIn, db: DbDep):
    return TarefaRepo(db).criar(dados)
```

O Pydantic converte o modelo SQLAlchemy na saída com `model_config = ConfigDict(from_attributes=True)` no schema `TarefaOut`.

Módulo 15 — AsyncIO para quem tem lógica forte

O problema que async resolve

Chamar uma API leva ~300 ms. Desses, ~299 ms são **espera de rede** — sua CPU parada. Isso é trabalho **I/O bound**. Async permite que, enquanto uma chamada espera, o programa avance outras. Um único processo, um único thread, milhares de operações de I/O simultâneas.

Não resolve trabalho **CPU bound** (cálculo pesado, parsing gigante) — para isso existem processos (`multiprocessing`). Regra:

Gargalo	Solução
Espera de rede/disco/banco (I/O bound)	asyncio
Cálculo intenso (CPU bound)	multiprocessing / worker
Poucas chamadas, script simples	síncrono mesmo — não complique

async / await

```
import asyncio
import httpx

async def buscar(client: httpx.AsyncClient, url: str) -> dict:
    r = await client.get(url)          # await = "libere o loop enquanto espero"
    r.raise_for_status()
    return r.json()

async def main() -> None:
    async with httpx.AsyncClient(timeout=10) as client:
        dados = await buscar(client, "https://api.github.com/repos/wazuh/wazuh")
        print(dados["stargazers_count"])

asyncio.run(main())                  # ponto de entrada – um por programa
```

Regras do jogo:

1. `async def` cria uma corrotina; chamá-la **não executa** — retorna um objeto aguardável.
2. `await` só existe dentro de `async def`.
3. Código bloqueante (`requests`, `time.sleep`, driver de banco síncrono) dentro de async **trava o loop inteiro** — use `httpx async`, `asyncio.sleep`, `drivers async`.

tasks e gather — o paralelismo de verdade

```
async def main() -> None:
    async with httpx.AsyncClient(timeout=10) as client:
        # sequencial disfarçado: 20 awaits em fila
        # for url in urls: await buscar(client, url)

        # concorrente: dispara tudo, espera tudo
        resultados = await asyncio.gather(
            *[buscar(client, url) for url in urls],
            return_exceptions=True,      # falha de uma não derruba as outras
        )
    for url, res in zip(urls, resultados):
        if isinstance(res, Exception):
            logger.warning("%s falhou: %s", url, res)
```

Exemplo com medição: 20 APIs simultâneas

```
import asyncio, time
import httpx, requests

URLS = [f"https://httpbin.org/delay/1?i={i}" for i in range(20)] # 1s cada

def versao_sincrona() -> float:
    inicio = time.perf_counter()
    with requests.Session() as s:
        for url in URLS:
            s.get(url, timeout=15)
    return time.perf_counter() - inicio

async def versao_async() -> float:
    inicio = time.perf_counter()
    async with httpx.AsyncClient(timeout=15) as c:
        await asyncio.gather(*[c.get(u) for u in URLS])
    return time.perf_counter() - inicio

print(f"Síncrono: {versao_sincrona():.1f}s") # ~20.5s (20 x 1s em fila)
print(f"Async: {asyncio.run(versao_async()):.1f}s") # ~1.3s (todas juntas)
```

Resultado típico: 20,5s → 1,3s. Mesma máquina, mesmo código de negócio — só mudou quem espera.

Controlando a concorrência (não derrube a API alheia)

```
sem = asyncio.Semaphore(10) # no máximo 10 simultâneas

async def buscar_limitado(client, url):
    async with sem:
        return await buscar(client, url)
```

Quando usar / quando NÃO usar

Use async: consumir dezenas/centenas de endpoints, webhooks de alto volume, WebSockets, APIs FastAPI com muito I/O externo.

Não use `async`: scripts de automação com meia dúzia de chamadas, jobs de CPU, quando as bibliotecas envolvidas são síncronas (misturar é a maior fonte de bugs). O FastAPI aceita rotas `def` (síncronas) e as executa num threadpool — uma rota síncrona bem escrita é melhor que uma rota `async` que bloqueia o loop.

Exercício rápido

Adapte o coletor: dado `agentes: list[str]`, consulte `GET /agents/{id}/status` para todos com no máximo 5 conexões simultâneas e retorne `{id: status}` ignorando os que falharem.

Módulo 16 — Autenticação e segurança

Hash de senha com bcrypt

Senha **nunca** é armazenada nem criptografada — é *hasheada* (irreversível, com salt automático):

```
pip install "passlib[bcrypt]"
```

```
from passlib.context import CryptContext

pwd = CryptContext(schemes=["bcrypt"], deprecated="auto")

hash_ = pwd.hash("S3nh@Forte")           # $2b$12$Xk9... (salt embutido, sempre diferente)
pwd.verify("S3nh@Forte", hash_)          # True
pwd.verify("errada", hash_)               # False
```

Você já viu isso por dentro: é o mesmo bcrypt do `internal_users` do OpenSearch que você corrigiu com `hash.sh` — mesmo algoritmo, mesma lógica de verificação.

JWT

Um JWT é `header.payload.assinatura` em base64. O servidor assina com uma chave secreta; qualquer um pode **ler** o payload, mas ninguém sem a chave consegue **forjar** a assinatura. Por isso: nada de dado sensível no payload, e `exp` (expiração) sempre.

```
pip install pyjwt
```

```
from datetime import datetime, timedelta, timezone
import jwt

SECRET = os.environ["JWT_SECRET"]          # 32+ bytes aleatórios; NUNCA no código
ALG = "HS256"

def criar_token(sub: str, minutos: int = 30) -> str:
    agora = datetime.now(timezone.utc)
    return jwt.encode(
        {"sub": sub, "iat": agora, "exp": agora + timedelta(minutes=minutos)},
        SECRET, algorithm=ALG,
    )

def decodificar(token: str) -> dict:
    return jwt.decode(token, SECRET, algorithms=[ALG])
    # Lança jwt.ExpiredSignatureError / jwt.InvalidTokenError
```

OAuth2 Password Flow no FastAPI

```
# app/routers/auth.py
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

router = APIRouter(tags=["auth"])
oauth2 = OAuth2PasswordBearer(tokenUrl="/login")

@router.post("/login")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()], db: DbDep):
    usuario = db.scalar(select(Usuario).where(Usuario.email == form.username))
    if usuario is None or not pwd.verify(form.password, usuario.senha_hash):
        raise HTTPException(401, "Credenciais inválidas",
                            headers={"WWW-Authenticate": "Bearer"})
    return {"access_token": criar_token(str(usuario.id)), "token_type": "bearer"}

def usuario_atual(token: Annotated[str, Depends(oauth2)], db: DbDep) -> Usuario:
    try:
        payload = decodificar(token)
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, "Token expirado")
    except jwt.InvalidTokenError:
        raise HTTPException(401, "Token inválido")
    usuario = db.get(Usuario, int(payload["sub"]))
    if usuario is None:
        raise HTTPException(401, "Usuário não existe")
    return usuario

UserDep = Annotated[Usuario, Depends(usuario_atual)]
```

Proteger qualquer rota agora é um parâmetro:

```
@router.get("/tarefas", response_model=list[TarefaOut])
def listar(usuario: UserDep, db: DbDep): # 401 automático sem token válido
    return TarefaRepo(db).listar_do_usuario(usuario.id)
```

Bônus: o Swagger ganha o botão **Authorize** — você faz login e testa rotas protegidas pela própria documentação.

Refresh token (conceito)

Access token curto (15–30 min) limita o estrago de um vazamento. Para não forçar login a cada meia hora: emita junto um **refresh token** longo (7–30 dias), armazenado no servidor (tabela com hash, revogável). O cliente troca o refresh por um novo access no endpoint `/refresh`. Se um refresh vazar, você o revoga no banco — coisa que não dá para fazer com JWT puro.

Secrets e variáveis de ambiente

- Gere: `python -c "import secrets; print(secrets.token_urlsafe(48))"`
- `.env` no `.gitignore`; produção usa secrets do orquestrador (Docker/K8s) ou cofre.
- Rotacione se houver suspeita de vazamento; logs **nunca** imprimem token/senha.

- Compare segredos com `secrets.compare_digest()` (resistente a timing attack) — vai aparecer no webhook do Módulo 22.

Rate limit (conceito)

Proteja `/login` de força bruta e a API de abuso. Em uma instância, `slowapi` resolve (`5/minute` no login); em produção maior, o padrão é limitar no Nginx/gateway ou com contadores no Redis. Resposta correta ao exceder: `429` + header `Retry-After`.

Projeto: adicionar login à API de tarefas

1. Model `Usuario` (email único, senha_hash) + migration Alembic.
2. Script/endpoint de criação de usuário (hash na entrada, nunca logue a senha).
3. Rotas de tarefas recebem `usuario: UserDep` e filtram por `usuario.id`.
4. Teste no Swagger: sem token → 401; com token → 200.

Módulo 17 — Testes que importam

pytest em 60 segundos

```
pip install pytest
```

```
# tests/test_importador.py – teste é função test_*, verificação é assert
from importador import parse_valor, limpar_documento

def test_parse_valor_formato_brasileiro():
    assert parse_valor("1.234,56") == 1234.56

def test_limpar_documento_remove_mascara():
    assert limpar_documento("123.456.789-01") == "12345678901"

def test_parse_valor_invalido():
    import pytest
    with pytest.raises(ValueError):
        parse_valor("abc")
```

```
pytest          # descobre e roda tudo em tests/
pytest -x -v     # para no primeiro erro, verboso
pytest -k "valor" # roda só testes com "valor" no nome
```

Fixtures — setup reutilizável

```
# tests/conftest.py (fixtures aqui ficam disponíveis para todos os testes)
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.core.database import Base

@pytest.fixture
def db():
    """Banco SQLite em memória, limpo a cada teste."""
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    sessao = sessionmaker(bind=engine)()
    yield sessao          # o teste roda aqui
    sessao.close()        # teardown

@pytest.fixture
def cliente_exemplo(db):
    c = Cliente(nome="Teste", documento="12345678901")
    db.add(c); db.commit()
    return c
```

```
def test_criar_titulo(db, cliente_exemplo):          # fixtures viram parâmetros
    repo = TituloRepo(db)
    t = repo.criar(cliente_exemplo.id, valor=100.0, vencimento=amanha)
    assert t.id is not None
    assert t.pago is False
```

TestClient do FastAPI

Testa a API inteira **sem subir servidor**:

```
# tests/conftest.py
from fastapi.testclient import TestClient
from app.main import app
from app.core.database import get_db

@pytest.fixture
def client(db):
    app.dependency_overrides[get_db] = lambda: db      # injeta o banco de teste
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()

@pytest.fixture
def token(client):
    client.post("/usuarios", json={"email": "t@t.com", "senha": "Secreta1!"})
    r = client.post("/login", data={"username": "t@t.com", "password": "Secreta1!"})
    return r.json()["access_token"]
```

Os três testes pedidos

```
# tests/test_api.py

def test_login_com_senha_errada_retorna_401(client):
    client.post("/usuarios", json={"email": "a@a.com", "senha": "Correta1!"})
    r = client.post("/login", data={"username": "a@a.com", "password": "errada"})
    assert r.status_code == 401

def test_criar_tarefa_autenticado(client, token):
    r = client.post("/tarefas",
                    json={"titulo": "Atualizar Chrome nos 500 agentes",
                          "prioridade": 5},
                    headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 201
    corpo = r.json()
    assert corpo["id"] > 0
    assert corpo["titulo"].startswith("Atualizar")

def test_erro_de_validacao_retorna_422(client, token):
    r = client.post("/tarefas",
                    json={"titulo": "ab", "prioridade": 99},      # ambos inválidos
                    headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 422
    campos_com_erro = {e["loc"][-1] for e in r.json()["detail"]}
    assert campos_com_erro == {"titulo", "prioridade"}
```

Mock de APIs externas

Seu teste não pode depender do gateway de pagamento estar no ar:

```
# pip install respx (mock específico para httpx)
import respx
from httpx import Response

@respx.mock
def test_criar_cobranca_trata_erro_500():
    respx.post("https://api.pag.com/v1/charges").mock(
        return_value=Response(500, json={"error": "internal"})
    )
    with pytest.raises(ErroIntegracao):
        api.criar_cobranca("12345678901", 19990)

@respx.mock
def test_criar_cobranca_sucesso():
    rota = respx.post("https://api.pag.com/v1/charges").mock(
        return_value=Response(201, json={"id": "ch_1", "status": "pending"})
    )
    resultado = api.criar_cobranca("12345678901", 19990)
    assert resultado["id"] == "ch_1"
    assert rota.called  # garante que a chamada aconteceu
```

Para código com `requests`, o equivalente é a lib `responses`; para mock genérico de funções, `unittest.mock.patch`.

O que testar (e o que não)

Priorize, nesta ordem: **regras de negócio** (cálculo de multa, validação de documento), **contratos da API** (status codes, formato de resposta, autenticação) e **integrações com falha simulada** (timeout, 500, JSON inválido). Não perca tempo testando o framework (o FastAPI já testa o próprio roteamento) nem getters triviais.

Exercício rápido

Escreva `test_listar_tarefas_de_outro_usuario_nao_aparece`: crie dois usuários, uma tarefa para cada, e garanta que o GET `/tarefas` de um não retorna a tarefa do outro.

Módulo 18 — Logs, observabilidade e debugging

logging estruturado

O objetivo: logs que uma máquina consegue indexar (você, como admin de SIEM, sabe exatamente por quê). Formato chave=valor ou JSON:

```
import json, logging, sys

class JsonFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        dados = {
            "ts": self.formatTime(record, "%Y-%m-%dT%H:%M:%S%z"),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
        }
        if record.exc_info:
            dados["exc"] = self.formatException(record.exc_info)
        if hasattr(record, "extra_data"):
            dados.update(record.extra_data)
        return json.dumps(dados, ensure_ascii=False)

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JsonFormatter())
logging.basicConfig(level=logging.INFO, handlers=[handler])

logger = logging.getLogger("api.cobrancas")
logger.info("título baixado", extra={"extra_data": {
    "titulo_id": 873, "valor": 199.90, "metodo": "pix"}})
# {"ts": "2026-07-29T14:03:22-0300", "level": "INFO", ...,
#  "titulo_id": 873, "valor": 199.9, "metodo": "pix"}
```

Alternativa sem código: `loguru` (`from loguru import logger; logger.info(...)`) já vem com formatação rica, rotação de arquivo e `serialize=True` para JSON.

Em containers, **logue para stdout** — quem coleta é o Docker/agente de log, não o seu app.

Níveis de log — critério prático

Nível	Use para	Exemplo
DEBUG	Detalhe de diagnóstico	payload completo, SQL gerado
INFO	Eventos de negócio normais	"título 873 baixado via pix"
WARNING	Anormal, mas contornado	"retry 2/3 no gateway"
ERROR	Operação falhou	"webhook rejeitado: assinatura inválida"
CRITICAL	Sistema comprometido	"sem conexão com o banco"

Produção roda em INFO; investigação liga DEBUG por env var (`LOG_LEVEL=DEBUG`), nunca por deploy.

traceback

```
try:
    processar(evento)
except ErroIntegracao:
    logger.exception("falha ao processar evento %s", evento_id)
```

`logger.exception` = `logger.error` + stack trace completo. Leia tracebacks **de baixo para cima**: a última linha é o erro; a primeira linha "sua" (do seu código, não da lib) é onde investigar. Com `raise ... from e` (Módulo 7), você vê a cadeia de causas inteira.

Debug no VS Code

Aposente o `print()`. Crie `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "FastAPI",
      "type": "debugpy",
      "request": "launch",
      "module": "uvicorn",
      "args": ["app.main:app", "--reload"],
      "envFile": "${workspaceFolder}/.env",
      "justMyCode": true
    }
  ]
}
```

F5 sobe a API sob o debugger. Breakpoint na rota, chamada pelo Swagger, e você inspeciona variáveis, avalia expressões no Debug Console e avança linha a linha (F10/F11). Breakpoints condicionais (clique direito → "cliente_id == 42") economizam horas em loops grandes. Alternativa rápida no terminal: `breakpoint()` no código abre o pdb.

Métricas (conceito)

Logs contam **o que aconteceu**; métricas contam **quanto/quão rápido**. As quatro de ouro para uma API: taxa de requisições, taxa de erro (5xx), latência (p50/p95/p99) e saturação (CPU/RAM/pool de conexões). O padrão do ecossistema é expor `/metrics` no formato Prometheus (`pip install prometheus-fastapi-instrumentator` faz isso em 2 linhas) e visualizar no Grafana — stack que você já conhece do mundo de monitoramento.

Healthcheck endpoint

```
from sqlalchemy import text

@app.get("/health", tags=["infra"])
def health(db: DbDep):
    try:
        db.execute(text("SELECT 1"))
        banco = "ok"
    except Exception:
        logger.exception("healthcheck: banco inacessível")
        raise HTTPException(503, "database unavailable")
    return {"status": "ok", "database": banco, "version": app.version}
```

É esse endpoint que o Docker (`HEALTHCHECK`), o load balancer e o seu monitoramento vão consultar. Regra: healthcheck **leve** (nada de varrer tabelas) e sem autenticação.

Módulo 19 — Docker para APIs Python

Você já domina Docker — aqui está só o específico de Python.

Dockerfile

```
FROM python:3.12-slim

# Logs sem buffer (aparecem em tempo real) e sem .pyc
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# 1º: dependências (camada cacheada – só reinstala se requirements mudar)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2º: código (muda sempre, camada barata)
COPY app/ ./app/
COPY migrations/ ./migrations/
COPY alembic.ini .

# Usuário não-root (você sabe por quê)
RUN useradd --create-home apiuser
USER apiuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD python -c "import httpx; httpx.get('http://localhost:8000/health',
    timeout=2).raise_for_status()"

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

A ordem `requirements.txt` antes do código é o detalhe que separa builds de 3 minutos de builds de 8 segundos.

.dockerignore

```
.env/
__pycache__/
*.pyc
.git/
.env                # NUNCA dentro da imagem
tests/
*.db
uploads/
```

docker compose: PostgreSQL + API

```
# compose.yaml
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: cobrancas
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d cobrancas"]
      interval: 5s
      retries: 10

  api:
    build: .
    environment:
      DATABASE_URL: postgresql+psycopg://app:${POSTGRES_PASSWORD}@db:5432/cobrancas
      JWT_SECRET: ${JWT_SECRET}
      LOG_LEVEL: INFO
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy      # só sobe com o banco pronto
    command: >
      sh -c "alembic upgrade head &&
        uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2"

volumes:
  pgdata:
```

```
# .env (lido pelo compose — fora do Git)
POSTGRES_PASSWORD=troque-isto
JWT_SECRET=gere-com-secrets.token_ursafe

docker compose up -d --build
docker compose logs -f api
curl http://localhost:8000/health
```

Pontos de atenção: o hostname do banco é o **nome do serviço** (`db`); as migrations rodam no arranque (`alembic upgrade head`) — em ambientes com múltiplas réplicas, mova isso para um job separado.

Módulo 20 — Deploy rápido

Opção 1: Render / Railway (mais rápido)

PaaS: você conecta o repositório GitHub e eles buildam do Dockerfile.

1. Suba o projeto (com Dockerfile) para o GitHub.
2. No painel: *New* → *Web Service* → *seu repo*. Detecta o Dockerfile sozinho.
3. Provisione o PostgreSQL gerenciado da própria plataforma e copie a `DATABASE_URL` gerada.
4. Configure as env vars (`DATABASE_URL` , `JWT_SECRET` , `LOG_LEVEL`).
5. Deploy automático a cada push na branch principal. HTTPS incluso.

Ideal para MVP e projetos internos. Limitação: menos controle de rede/recursos, e o free tier hiberna.

Opção 2: VPS Ubuntu (mais controle — seu território)

```
# 1. Base
sudo apt update && sudo apt install -y docker.io docker-compose-v2 nginx certbot python3-
certbot-nginx
sudo usermod -aG docker $USER

# 2. Código
git clone https://github.com/voce/api-cobrancas.git /opt/api-cobrancas
cd /opt/api-cobrancas
cp .env.example .env && nano .env      # secrets de produção

# 3. Subir
docker compose up -d --build
```

systemd — garantir que sobe com a máquina

```
# /etc/systemd/system/api-cobrancas.service
[Unit]
Description=API de Cobrancas
Requires=docker.service
After=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/api-cobrancas
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=300

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now api-cobrancas
```

Ngix — reverse proxy + TLS (visão geral)

```
# /etc/nginx/sites-available/api
server {
    server_name api.suaempresa.com.br;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        client_max_body_size 10m;          # uploads de comprovante
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/api /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d api.suaempresa.com.br    # HTTPS automático
```

Ngix na frente resolve TLS, compressão, limite de corpo e (se quiser) rate limit — deixando o uvicorn atrás só com HTTP puro. Exatamente o padrão que você já usa com dashboards atrás de proxy.

Checklist de produção

- [] `DEBUG` desligado; docs (`/docs`) atrás de auth ou desabilitadas se a API for pública
- [] Secrets fora do Git e fora da imagem
- [] Backup do volume do PostgreSQL (`pg_dump` agendado)
- [] Logs coletados (você pode inclusive apontar o agente do seu SIEM para eles)
- [] `/health` monitorado
- [] Firewall: só 80/443 públicos; 8000 e 5432 fechados

Módulo 21 — Bibliotecas que você realmente usará

Biblioteca	Para que serve	Observação 2026
requests	HTTP simples e síncrono	Padrão em scripts; maduro e estável
httpx	HTTP moderno, sync + async, HTTP/2	Escolha para projetos novos
fastapi	APIs REST com validação e docs automáticas	O framework backend dominante
pydantic	Validação e serialização de dados	v2 (núcleo em Rust, muito rápido)
sqlalchemy	ORM / acesso a banco	Use a sintaxe 2.x (<code>select()</code> , <code>Mapped</code>)
alembic	Migrations de schema	Versionamento de banco no Git
pytest	Testes	+ <code>respx</code> para mockar httpx
python-dotenv	Carregar <code>.env</code>	Ou <code>pydantic-settings</code> (tipado)
loguru	Logs bonitos e estruturados sem config	Alternativa ao logging da stdlib
pandas	Análise/transformação de dados e CSV grandes	P/ CSVs simples, <code>csv</code> da stdlib basta
tenacity	Retry com backoff via decorator	<code>@retry(wait=wait_exponential(...))</code>
aiofiles	Leitura/escrita de arquivos em código async	Evita bloquear o event loop
uvicorn	Servidor ASGI que roda o FastAPI	<code>--workers N</code> em produção

Complementos que aparecem cedo na prática: **psycopg** (driver PostgreSQL), **passlib** (bcrypt), **pyjwt** (tokens), **typer** (CLIs rápidas), **apscheduler** (agendamento interno de tarefas).

Módulo 22 — Integrações reais (o mais importante)

Integração 1 — Consumir API REST externa (auth + paginação + retry)

Cliente completo, pronto para adaptar a qualquer API (gerenciador de endpoints, gateway de pagamento, ERP):

```

"""cliente_api.py – padrão profissional de consumo de API externa."""
import logging
import httpx
from tenacity import (retry, retry_if_exception, stop_after_attempt,
                      wait_exponential)

logger = logging.getLogger(__name__)

def _erro_temporario(exc: BaseException) -> bool:
    """Retry só no que pode se resolver sozinho."""
    if isinstance(exc, (httpx.TimeoutException, httpx.ConnectError)):
        return True
    if isinstance(exc, httpx.HTTPStatusError):
        return exc.response.status_code in (429, 500, 502, 503, 504)
    return False
    # 401/404/422: retry não ajuda

class ApiExterna:
    def __init__(self, base_url: str, usuario: str, senha: str):
        self._client = httpx.Client(base_url=base_url, timeout=15.0, verify=True)
        self._auth = (usuario, senha)
        self._token: str | None = None

    def _autenticar(self) -> None:
        r = self._client.post("/security/user/authenticate", auth=self._auth)
        r.raise_for_status()
        self._token = r.json()["data"]["token"]
        self._client.headers["Authorization"] = f"Bearer {self._token}"
        logger.info("Token renovado")

    @retry(retry=retry_if_exception(_erro_temporario),
          stop=stop_after_attempt(4),
          wait=wait_exponential(multiplier=1, max=30), # 1s, 2s, 4s... máx 30s,
          reraise=True)
    def _get(self, rota: str, **params) -> dict:
        if self._token is None:
            self._autenticar()
        r = self._client.get(rota, params=params)
        if r.status_code == 401:
            # token expirou no meio do lote
            self._autenticar()
            r = self._client.get(rota, params=params)
        r.raise_for_status()
        return r.json()

    def listar_todos(self, rota: str, **filtros) -> list[dict]:
        """Paginação offset/limit até esgotar."""
        itens, offset, limite = [], 0, 500
        while True:
            corpo = self._get(rota, offset=offset, limit=limite, **filtros)
            lote = corpo["data"]["affected_items"]
            itens.extend(lote)
            logger.debug("Página offset=%d: %d itens", offset, len(lote))
            if len(lote) < limite:
                return itens
            offset += limite

# Uso
api = ApiExterna("https://gerenciador:55000", os.environ["API_USER"],
                 os.environ["API_PASS"])
desatualizados = [

```

```

    a for a in api.listar_todos("/agents", status="active")
    if a.get("version", "") < "v4.9.0"
]
logger.info("%d agentes desatualizados", len(desatualizados))

```

Os quatro pilares presentes: **autenticação com renovação, paginação até o fim, retry exponencial só em erros temporários e timeout em tudo.**

Integração 2 — Receber webhook

Regras de ouro de webhook: **valide a assinatura, responda em < 5s, processe depois, seja idempotente** (o provedor REENVIA eventos).

```

"""app/routers/webhooks.py"""
import hashlib, hmac, os
from fastapi import APIRouter, BackgroundTasks, Header, HTTPException, Request

router = APIRouter(prefix="/webhooks", tags=["webhooks"])
SEGREDO = os.environ["WEBHOOK_SECRET"].encode()

def assinatura_valida(corpo: bytes, recebida: str) -> bool:
    esperada = hmac.new(SEGREDO, corpo, hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperada, recebida) # compara em tempo constante

@router.post("/pagamentos", status_code=200)
async def receber_pagamento(
    request: Request,
    bg: BackgroundTasks,
    x_signature: str = Header(...),
):
    corpo = await request.body() # bytes BRUTOS p/ assinatura

    # 1. Autenticidade
    if not assinatura_valida(corpo, x_signature):
        raise HTTPException(401, "Assinatura inválida")

    # 2. Estrutura (Pydantic do Módulo 13)
    try:
        evento = EventoWebhook.model_validate_json(corpo)
    except ValidationError:
        logger.warning("Webhook malformatado: %.300s", corpo)
        return {"status": "ignored"} # 200 mesmo assim: reenvio não vai corrigir

    # 3. Idempotência: evento já visto? Confirma e sai.
    if ja_processado(evento.event_id):
        return {"status": "duplicate"}

    # 4. Responde JÁ; processa depois
    bg.add_task(processar_evento, evento)
    return {"status": "accepted"}

def processar_evento(evento: EventoWebhook) -> None:
    marcar_processado(evento.event_id)
    if evento.type == "charge.paid":
        baixar_titulo(evento.data.document, evento.data.amount / 100)
        enviar_recibo_whatsapp(evento.data.document)

```

Para testar localmente com provedores reais, exponha sua porta com um túnel (`cloudflared tunnel` ou similar) e aponte o webhook de sandbox para ele.

Integração 3 — Fluxo completo: CSV → validação → PostgreSQL → API → JSON

O pipeline que une tudo: o CSV de títulos entra, o Pydantic valida, o SQLAlchemy persiste, e a API externa gera as cobranças.

```

"""pipeline_cobranca.py – orquestra o fluxo completo."""
import csv, logging
from pathlib import Path
from pydantic import BaseModel, Field, ValidationError, field_validator

logger = logging.getLogger(__name__)

# — 1. Schema de entrada —————
class LinhaCSV(BaseModel):
    nome: str = Field(min_length=3)
    documento: str
    valor: float = Field(gt=0)
    vencimento: date

    @field_validator("documento")
    @classmethod
    def _doc(cls, v: str) -> str:
        v = re.sub(r"\D", "", v)
        if len(v) not in (11, 14):
            raise ValueError("documento inválido")
        return v

    @field_validator("valor", mode="before")
    @classmethod
    def _valor_br(cls, v):
        return float(str(v).replace(".", "").replace(",", ".")) \
            if isinstance(v, str) else v

# — 2. Ler e validar —————
def ler_csv(caminho: Path) -> tuple[list[LinhaCSV], int]:
    validos, erros = [], 0
    with caminho.open(encoding="utf-8-sig", newline="") as f:
        for n, linha in enumerate(csv.DictReader(f, delimiter=";", start=2)):
            try:
                validos.append(LinhaCSV.model_validate(linha))
            except ValidationError as e:
                erros += 1
                logger.warning("Linha %d rejeitada: %s", n,
                               "; ".join(x["msg"] for x in e.errors()))

    return validos, erros

# — 3. Persistir (upsert por documento+vencimento) —————
def persistir(db: Session, linhas: list[LinhaCSV]) -> list[Titulo]:
    novos = []
    for l in linhas:
        cliente = db.scalar(select(Cliente)
                             .where(Cliente.documento == l.documento))
        if cliente is None:
            cliente = Cliente(nome=l.nome, documento=l.documento)
            db.add(cliente)
            db.flush()
        existe = db.scalar(select(Titulo).where(
            Titulo.cliente_id == cliente.id,
            Titulo.vencimento == l.vencimento))
        if existe:
            logger.info("Título já existe: %s %s", l.documento, l.vencimento)
            continue
        titulo = Titulo(cliente_id=cliente.id, valor=l.valor,
                        vencimento=l.vencimento)

```

```

        db.add(titulo)
        novos.append(titulo)
    db.commit()
    return novos

# — 4. Gerar cobranças na API externa —————
def gerar_cobrancas(db: Session, api: ApiPagamentos,
                    titulos: list[Titulo]) -> dict:
    ok = falha = 0
    for t in titulos:
        try:
            cobranca = api.criar_cobranca(      # cliente com retry (Integração 1)
                documento=t.cliente.documento,
                valor_centavos=int(t.valor * 100),
                vencimento=t.vencimento.isoformat())
            t.cobranca_externa_id = cobranca["id"]
            ok += 1
        except ErroIntegracao:
            logger.exception("Falha na cobrança do título %d", t.id)
            falha += 1
    db.commit()
    return {"geradas": ok, "falhas": falha}

# — 5. Orquestração —————
def executar(caminho_csv: Path) -> dict:
    linhas, rejeitadas = ler_csv(caminho_csv)
    with SessionLocal() as db:
        novos = persistir(db, linhas)
        resultado = gerar_cobrancas(db, api_pagamentos, novos)
    resumo = {"lidas": len(linhas) + rejeitadas, "validas": len(linhas),
             "rejeitadas": rejeitadas, "novos_titulos": len(novos),
             **resultado}
    logger.info("Pipeline concluído: %s", resumo)
    return resumo          # a rota FastAPI devolve isto como JSON

```

Exponha na API para o time da farmácia usar sem terminal:

```

@router.post("/importacoes", status_code=202)
async def importar(arquivo: UploadFile, bg: BackgroundTasks, usuario: UserDep):
    destino = Path("uploads") / f"import_{uuid.uuid4()}.csv"
    destino.write_bytes(await arquivo.read())
    bg.add_task(executar, destino)
    return {"status": "processing", "arquivo": arquivo.filename}

```

Módulo 23 — Projeto final: Sistema de Cobranças e Lembretes

O projeto consolida **tudo**: clientes, contas a receber, upload de comprovante, JWT, PostgreSQL, logs, testes, Docker e Swagger. É diretamente aplicável à realidade de uma farmácia/oficina com clientes recorrentes.

Estrutura completa

```

cobrancas/
├── app/
│   ├── __init__.py
│   ├── main.py           # cria o app, middleware, routers
│   └── core/
│       ├── config.py     # Settings (pydantic-settings)
│       ├── database.py   # engine, SessionLocal, Base, get_db
│       ├── security.py   # bcrypt, JWT, usuario_atual
│       └── logging.py    # JsonFormatter / loguru
│   ├── models/
│       ├── usuario.py
│       ├── cliente.py
│       ├── conta.py      # ContaReceber
│       └── comprovante.py
│   ├── schemas/
│       ├── auth.py       # TokenOut, LoginIn
│       ├── cliente.py    # ClienteIn, ClienteOut
│       └── conta.py      # ContaIn, ContaOut, BaixaIn
│   ├── repositories/
│       ├── cliente_repo.py
│       └── conta_repo.py
│   ├── services/
│       ├── cobranca_service.py # regras: multa, baixa, saldo
│       └── lembrete_service.py # mensagens de vencimento
│   └── routers/
│       ├── auth.py       # POST /login
│       ├── clientes.py   # CRUD /clientes
│       ├── contas.py     # CRUD /contas, POST /contas/{id}/baixa
│       ├── comprovantes.py # POST /contas/{id}/comprovante
│       └── infra.py      # GET /health
├── migrations/           # Alembic
├── tests/
│   ├── conftest.py       # fixtures: db, client, token
│   ├── test_auth.py
│   ├── test_clientes.py
│   ├── test_contas.py
│   └── test_comprovantes.py
├── uploads/              # volume p/ comprovantes
├── .env.example
├── .dockerignore
├── .gitignore
├── alembic.ini
├── compose.yaml
├── Dockerfile
├── pyproject.toml
└── requirements.txt
  
```

Models principais

```
# app/models/conta.py
class ContaReceber(Base):
    __tablename__ = "contas_receber"

    id: Mapped[int] = mapped_column(primary_key=True)
    cliente_id: Mapped[int] = mapped_column(ForeignKey("clientes.id"), index=True)
    descricao: Mapped[str] = mapped_column(String(200))
    valor: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    vencimento: Mapped[date] = mapped_column(index=True)
    status: Mapped[str] = mapped_column(String(20), default="aberta")
    # aberta | paga | cancelada
    pago_em: Mapped[datetime | None]
    valor_pago: Mapped[Decimal | None] = mapped_column(Numeric(10, 2))
    criada_em: Mapped[datetime] = mapped_column(server_default=func.now())

    cliente: Mapped["Cliente"] = relationship(back_populates="contas")
    comprovantes: Mapped[list["Comprovante"]] = relationship(
        back_populates="conta", cascade="all, delete-orphan")
```

Note o `Decimal` / `Numeric` para dinheiro — float acumula erro de arredondamento; em valores financeiros, use sempre `Decimal`.

Regra de negócio central

```
# app/services/cobranca_service.py
MULTA = Decimal("0.02") # 2%
JUROS_DIA = Decimal("0.00033") # ~1% a.m.

def valor_atualizado(conta: ContaReceber, hoje: date | None = None) -> Decimal:
    hoje = hoje or date.today()
    if conta.status != "aberta" or hoje <= conta.vencimento:
        return conta.valor
    dias = (hoje - conta.vencimento).days
    return (conta.valor * (1 + MULTA + JUROS_DIA * dias)).quantize(Decimal("0.01"))

def baixar(db: Session, conta: ContaReceber, valor_pago: Decimal) -> ContaReceber:
    if conta.status == "paga":
        raise ContaJaPagaError(conta.id)
    conta.status = "paga"
    conta.pago_em = datetime.now(timezone.utc)
    conta.valor_pago = valor_pago
    db.commit()
    logger.info("conta baixada", extra={"extra_data": {
        "conta_id": conta.id, "valor_pago": str(valor_pago)}})
    return conta
```

Rotas (resumo do contrato)

```

POST    /login                                → {access_token}
POST    /clientes                            201    → ClienteOut
GET     /clientes?busca=&limit=                → [ClienteOut]
GET     /clientes/{id}                        → ClienteOut (404 se não existe)
POST    /contas                              201    → ContaOut
GET     /contas?status=aberta&vencidas=true → [ContaOut c/ valor_atualizado]
POST    /contas/{id}/baixa                    → ContaOut (409 se já paga)
POST    /contas/{id}/comprovante             201    → {arquivo} (PDF/JPG ≤ 5MB)
GET     /health                              → {status, database, version}

```

Todas as rotas (exceto `/login` e `/health`) exigem `usuario: UserDep`. O Swagger em `/docs` documenta e testa tudo com o botão Authorize.

Lembretes de vencimento (automação)

```

# app/services/lembrete_service.py
def contas_a_lembrar(db: Session, dias_antes: int = 3) -> list[ContaReceber]:
    alvo = date.today() + timedelta(days=dias_antes)
    return list(db.scalars(
        select(ContaReceber)
        .options(selectinload(ContaReceber.cliente))
        .where(ContaReceber.status == "aberta",
              ContaReceber.vencimento == alvo)))

def enviar_lembretes(db: Session, notificador: Notificador) -> int:
    contas = contas_a_lembrar(db)
    for c in contas:
        notificador.enviar(
            c.cliente.whatsapp,
            f"Olá {c.cliente.nome}! Sua conta '{c.descricao}' de "
            f"R$ {c.valor:.2f} vence em {c.vencimento:%d/%m}. "
            f"Pague pelo PIX e envie o comprovante. ")
    return len(contas)

```

Agende com cron no host (`0 9 * * * docker compose exec api python -m app.jobs.lembretes`) ou com APScheduler dentro do app.

Testes mínimos do projeto

`tests/` cobre: login ok/401, criar cliente (201 + documento duplicado 409), criar conta (validação 422), listar contas vencidas com valor atualizado, baixa (200 e 409 na segunda), upload de comprovante (201, tipo errado 415) e health (200). Com as fixtures do Módulo 17, são ~15 testes e menos de 200 linhas.

Subir tudo

```

cp .env.example .env      # preencha POSTGRES_PASSWORD e JWT_SECRET
docker compose up -d --build
docker compose exec api pytest -q      # rode os testes dentro do container
xdg-open http://localhost:8000/docs

```

Roteiro de construção sugerido (na ordem): config + database + health → models + migrations → auth → clientes → contas + regras → comprovantes → lembretes → testes → Docker. Cada etapa termina com a API subindo e o Swagger funcionando.

Módulo 24 — Plano de estudo de 7 dias (2h30/dia)

Dia	Conteúdo	Prática obrigatória (a maior parte do tempo)
1	Módulos 1-3: mentalidade, setup, fundamentos	Instale tudo, faça o parser de query string e reescreva um script shell seu em Python
2	Módulos 4-5: estruturas, strings, datas, JSON	Transforme um JSON real de API que você usa (relatório por grupo + datas locais)
3	Módulos 6-7, 10: arquivos, erros, HTTP	Importador de CSV + consumir uma API real com Bearer token, timeout e tratamento de erro
4	Módulos 11-13: REST, FastAPI, Pydantic	API de tarefas em memória: CRUD, validação, Swagger, routers
5	Módulo 14: SQLAlchemy + PostgreSQL	Persistir a API de tarefas; Alembic com 2 migrations; resolver um N+1 de propósito
6	Módulos 16-17: JWT, testes (+ 15 se sobrar)	Login + rotas protegidas + os 3 testes do Módulo 17 passando
7	Módulos 19-20, 22-23: Docker, integrações, projeto	<code>docker compose up</code> do projeto final com webhook fake e pipeline CSV

Regras do plano: **50 min teoria / 100 min código**, sem pular a prática; se atrasar, corte teoria e não prática; ao final de cada dia, commit no Git com o que funcionou.

As 20% de Python que dão 80% do resultado

Se o tempo apertar, priorize nesta ordem:

1. **dict, list e compreensões** — base de todo processamento de JSON
2. **Funções + kwargs** — reutilização em APIs e serviços
3. **Type hints** — destravam FastAPI e Pydantic
4. **JSON** — entrada e saída de tudo
5. **requests/httpx** — integração com o mundo
6. **FastAPI** — criar APIs profissionais rápido
7. **Pydantic** — validação nas fronteiras
8. **SQLAlchemy** — persistência em PostgreSQL
9. **AsyncIO** — concorrência de I/O
10. **pytest** — confiança para mudar código

O exercício de 90 minutos que mais acelera

Construa, sem copiar e colar, um serviço que: **consome uma API externa → valida com Pydantic → salva no PostgreSQL → expõe via FastAPI → protege com JWT**. São os cinco músculos centrais num único fluxo. Se travar em alguma etapa, você descobriu exatamente qual módulo reler.

Módulo 25 — Cheatsheets finais

Sintaxe essencial

```
# f-string          f"{valor:.2f}"  f"{data:%d/%m/%Y}"  f"{var=}"
# ternário          x = a if cond else b
# desempacotar      a, b = par; primeiro, *resto = lista
# comprehension     [f(x) for x in xs if cond]
#                  {k: f(v) for k, v in d.items()}
# mesclar dicts     novo = base | override
# enumerate/zip      for i, x in enumerate(xs, 1):  for a, b in zip(xs, ys):
# walrus             if (n := len(itens)) > 100: ...
# match              match ev["type"]: case "paid": ... case _: ...
# with               with open(p) as f, httpx.Client() as c: ...
```

requests

```
s = requests.Session()
s.headers.update({"Authorization": f"Bearer {tok}"})
r = s.get(url, params={"q": 1}, timeout=(3, 27)); r.raise_for_status()
r = s.post(url, json=payload, timeout=10)
r.status_code r.json() r.text r.content r.headers
s.post(url, files={"file": open("a.pdf", "rb")}, timeout=30)
```

HTTPX

```
with httpx.Client(base_url=B, headers=H, timeout=10) as c:
    c.get("/x", params=...); c.post("/x", json=...)

async with httpx.AsyncClient(base_url=B, timeout=10) as c:
    rs = await asyncio.gather(*[c.get(u) for u in urls])

# Erros: httpx.TimeoutException, httpx.ConnectError, httpx.HTTPStatusError
```

FastAPI

```
app = FastAPI(title="X")
router = APIRouter(prefix="/r", tags=["r"])
app.include_router(router)

@router.get("/{id}", response_model=Out)          # path param
def f(id: int, q: str = "", db: DbDep, u: UserDep): ...

@router.post("", status_code=201)                 # body = BaseModel
def g(dados: In, bg: BackgroundTasks): bg.add_task(fn, x)

raise HTTPException(404, "não encontrado")
# dev: fastapi dev main.py  prod: uvicorn app.main:app --workers 2
# docs: /docs  schema: /openapi.json
```

SQLAlchemy 2.x

```
class T(Base):
    __tablename__ = "t"
    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(80), index=True)
    dono_id: Mapped[int] = mapped_column(ForeignKey("u.id"))
    dono: Mapped["U"] = relationship(back_populates="ts")

db.add(obj); db.commit(); db.refresh(obj)
db.get(T, 1)
db.scalar(select(T).where(T.nome == "x"))
db.scalars(select(T).order_by(T.id).limit(10)).all()
select(T).options(selectinload(T.dono))          # evita N+1
db.delete(obj); db.commit()
# alembic revision --autogenerate -m "msg" && alembic upgrade head
```

AsyncIO

```
async def f(): return await coro()
asyncio.run(main())
rs = await asyncio.gather(*coros, return_exceptions=True)
async with asyncio.Semaphore(10): ...
await asyncio.sleep(1)          # nunca time.sleep em async
async with asyncio.timeout(5): ... # 3.11+
```

pytest

```
def test_x(): assert f(2) == 4
with pytest.raises(ValueError): f("a")

@pytest.fixture
def recurso(): r = criar(); yield r; r.fechar()

@pytest.mark.parametrize("entrada,esperado", [("1,5", 1.5), ("2", 2.0)])
def test_p(entrada, esperado): assert parse(entrada) == esperado

client = TestClient(app); client.get("/x", headers={"Authorization": f"Bearer {t}"})
app.dependency_overrides[get_db] = lambda: db_teste
# pytest -x -v -k "nome"
```

logging

```
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s")
logger.info("evento %s", x)          # lazy formatting, não f-string
logger.exception("falhou")          # dentro de except: msg + traceback
logging.getLogger("httpx").setLevel(logging.WARNING) # silencia lib verbosa
```

Apêndice A — Erros mais comuns de iniciantes técnicos

1. Default mutável em função

```
def add(item, lista=[]):          # a MESMA lista em todas as chamadas
    lista.append(item); return lista

def add(item, lista=None):        #
    lista = lista or []
    lista.append(item); return lista
```

2. Comparar com None usando ==

```
if resultado == None: ...        # funciona por acidente
if resultado is None: ...        # identidade, não igualdade
```

3. Capturar exceção larga demais

```
except Exception: pass          # engoliu o bug; você vai caçá-lo por horas
except httpx.TimeoutException:  # específico + log
    logger.warning("timeout em %s", url)
```

4. Modificar lista enquanto itera

```
for a in agentes:                # pula elementos
    if a["status"] == "never_connected": agentes.remove(a)

agentes = [a for a in agentes    # construa outra lista
            if a["status"] != "never_connected"]
```

5. Esquecer timeout em HTTP — `requests.get(url)` sem timeout trava para sempre. Sempre `timeout=.`

6. Misturar naive e aware datetime — `datetime.now()` vs `datetime.now(timezone.utc)`: comparar os dois lança `TypeError`. Padronize UTC aware.

7. Import circular — `services` importa `routers` que importa `services`. Sinal de camadas erradas: dependências fluem numa direção só (router → service → repo).

8. Float para dinheiro — `0.1 + 0.2 == 0.30000000000000004`. Use `Decimal` (ou centavos em int).

9. Rodar `python arquivo.py` dentro de pacote e tomar `ModuleNotFoundError` — use `python -m app.main` a partir da raiz.

10. Segredo commitado no Git — uma vez no histórico, considere vazado: rotacione a credencial (você sabe bem). `.env` no `.gitignore` desde o commit 1.

11. `if len(lista) > 0` — pythonico é `if lista:`. Igualmente: `if not lista:` para vazia.

12. Reimplementar o que o framework dá — validação manual de body no FastAPI, parsing manual de query string, SQL string concatenado (injeção!) em vez do ORM.

Apêndice B — Estrutura de projeto recomendada para produção

```

projeto/
├── app/
│   ├── main.py           # só bootstrap: app, middleware, include_router
│   ├── core/             # config, database, security, logging
│   ├── models/           # SQLAlchemy (1 arquivo por agregado)
│   ├── schemas/          # Pydantic entrada/saída (nunca reuse models aqui)
│   ├── repositories/     # todo SQL vive aqui
│   ├── services/         # regra de negócio pura (fácil de testar)
│   ├── routers/          # HTTP fino: valida, chama service, responde
│   └── jobs/             # scripts agendados (python -m app.jobs.x)
├── migrations/           # alembic
├── tests/                # espelha a estrutura de app/
├── .env.example          # documenta TODAS as env vars, sem valores reais
├── compose.yaml
├── Dockerfile
├── pyproject.toml        # deps + config de ruff/pytest
└── README.md            # como rodar em 5 comandos

```

Princípios: dependências fluem router → service → repository → banco (nunca ao contrário); schemas Pydantic são o contrato externo e models SQLAlchemy o interno — não misture; tudo que muda por ambiente vem de env var; se um arquivo passa de ~300 linhas, divida.

Apêndice C — Como ler documentação Python rapidamente

1. **Comece pelo Quickstart/Tutorial da lib, não pela API reference.** 10 minutos de quickstart > 1h de reference.
2. **Na reference, leia a assinatura e os exemplos; pule a prosa.** A assinatura tipada (`def get(url, *, params=None, timeout=...)`) diz quase tudo.
3. **Use o Pylance como documentação viva:** `Ctrl+Espaço` lista o que existe; passar o mouse mostra tipos e docstring; `F12` abre o código-fonte da lib — que muitas vezes explica mais que o site.
4. **No REPL, `dir(obj)` e `help(obj.metodo)`** respondem 80% das dúvidas sem abrir o navegador.
5. **Confira a versão!** Muito conteúdo na internet é Pydantic v1 ou SQLAlchemy 1.4. Cheque a data e a versão na doc oficial (procure "v2", "2.0 style").
6. **Fontes canônicas:** docs.python.org (stdlib), fastapi.tiangolo.com, docs.pydantic.dev, docs.sqlalchemy.org — todas têm busca própria melhor que buscador genérico.
7. **Leia o changelog antes de atualizar** dependência de produção — igual você faz com qualquer software de infra.

Fim do curso. Agora feche o PDF e abra o VS Code: o Dia 1 do plano de 7 dias começa com
`python3 -m venv .venv`.